

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
=====
=====
```

THE GEOMETRIC LANGUAGE MACHINE, THIRD GENERATION

Reasoning inside the Monster:

$\Lambda / 2\Lambda$ as the index set of meaning, the extraspecial group $2^{(1+24)}$ as its composition law, and the 196,884-dimensional Griess algebra as the place where two concepts are compared

An operational paper. Reading it is the documentation; running it is the verification. Every numbered claim below is checked by the code in this file, against the modules it describes.

`python3 glm3_paper.py` full run (about 2 min)

`python3 glm3_paper.py --quick` skips the sweeps (about 95 s)

`python3 glm3_paper.py --json` writes results/glm3_results.json only

Companion implementation: `glm3_reasoner.py`

Test suite: `test_glm3.py`

```
=====
=====
```

ABSTRACT

The two earlier generations of this system named the Monster group and did not use it. GLM-1 built the Golay code and M24 and stopped there; GLM-2 built the Leech lattice and Co₀, quoted the ledger $196,884 = 300 + 98,280 + 98,304$, and then modelled the algebra above it by proxy — a Jordan algebra of symmetric matrices and a Matsuo algebra of a 3-transposition group, both of them honest constructions but neither of them the Griess algebra, and neither of them containing a single concept of the register.

Section 8 of the GLM-2 paper says so in as many words.

This third generation removes the proxy. The chain

meaning $\rightarrow$ Leech point $\rightarrow$ the 2-adic stack of $\Lambda/2\Lambda$
 $\rightarrow Q = 2^{(1+24)}$ $\rightarrow$ the even part of the Griess algebra

is built end to end, exactly, in integers and Fractions, and the register of 660 concepts is carried all the way along it. Nothing on the way is quoted from the literature: the quadratic form, its Witt type, the class census, the cocycle of the extraspecial group, its 4096-dimensional representation, the four structure constants of the algebra, the axis spectrum, the fusion law and the Norton-Sakuma types are all COMPUTED here and checked against each other.

Ten things are new, and each is a claim in this paper.

(i) $\Lambda / 2\Lambda$ IS THE INDEX SET OF MEANING, NOT Λ . The Monster does not act on the Leech lattice. It acts on structures indexed by the 2^{24} classes of $\Lambda / 2\Lambda$, on which the Leech norm descends to an

F_2 quadratic form $q(\lambda) = (\lambda.\lambda)/16 \pmod 2$ with polarisation $B(\lambda, \mu) = (\lambda.\mu)/8 \pmod 2$. We compute the Witt decomposition of that form: twelve hyperbolic planes and no anisotropic part, so the form is of PLUS type, and the singular classes number $2^{23} + 2^{11} = 8,390,656$. The class census $1 + 98,280 + 8,386,560 + 8,292,375 = 2^{24}$ closes, and the type-3 classes are exactly the non-singular ones — which gives an $O(1)$ test for the type of a class where GLM-2 needed a lattice decoder. Section 3.

(ii) A CONCEPT IS A STACK OF MONSTER ADDRESSES — the multi-MOG-cube. A single reduction mod 2 is far too coarse to carry meaning: the 660 concepts of the register land on nine distinct classes. That is not a defect of the

Monster, it is the same mod-2 ceiling GLM-1 diagnosed, met again one level up. The fix is the same as GLM-1's: do not reduce, EXPAND. The k -th binary digit plane of the 24 Leech-basis coordinates is a class of $\Lambda / 2\Lambda$, hence a Monster address; ten planes rebuild the carrier exactly, so the word of ten addresses is a faithful encoding of the meaning, and every question about a concept becomes ten questions inside the Monster. This is the multi-MOG-cube in the basis view; Section 6 shows it is also the multi-MOG-cube in the ambient view, where plane 1 of a Leech point is forced to be a Golay codeword and the planes above it are forced to satisfy a mod-8 sum condition — the Leech congruences ARE the stack. Sections 6 and 7.

(iii) COMPOSITION OF MEANINGS IS MULTIPLICATION IN AN EXTRASPECIAL GROUP.

From the quadratic form alone we build $Q = 2^{(1+24)}_+$, of order 2^{25} , by finding a symplectic singular basis of $\Lambda / 2\Lambda$ and using it to define an explicit cocycle f with $f(u,u) = q(u)$ and $f(u,v) + f(v,u) = B(u,v)$. The group relations $x_u^2 = z^{q(u)}$ and $[x_u, x_v] = z^{B(u,v)}$ are checked, the involution count $2^{24} + 2^{12}$ is verified, and the 4096-dimensional Schrodinger representation is built and shown to be a faithful homomorphism in which the centre acts as -1 . Because GLM composition adds carriers and plane 0 is additive, composing two concepts multiplies their group elements: $x_u x_v = z^{f(u,v)} x_{\{u+v\}}$. The GLM product law and the Monster's 2B-centraliser group law are the same law. Section 4.

(iv) THE EVEN PART OF THE GRIESS ALGEBRA IS BUILT, WITH ITS CONSTANTS

DERIVED. We construct the 98,580-dimensional commutative algebra $300 + 98,280$ exactly: a symmetric 24×24 rational matrix part A , and a sparse part spanned by one basis vector b_λ per type-2 class. The four structure constants and the two form constants are not quoted from Griess or Conway: they are pinned down here by N -equivariance, by the requirement that the identity act as the identity, by the Frobenius property, and finally by the fusion law itself. The result is that $a_\lambda = (1/8) P_\lambda \pm (1/2) b_\lambda$ is idempotent of norm one, that

ad(a) has spectrum exactly $\{1, 0, 1/4, 1/32\}$ with dimensions $1 / 49,152 / 2,323 / 47,104$ summing to 98,580, and that ALL SIX non-trivial Ising fusion rules hold, including the two graded ones. These are Majorana 2A axes of the Monster, 98,280 of them, and the register's concepts sit on them. Section 5.

(v) THE MONSTER DOES THE REASONING. The companion implementation glm3_reasoner.py answers the GLM-2 questions unchanged — audit, solve, convert, identify, repair — and then answers new ones that only the Monster layer can answer. Two concepts have a RELATION WORD, ten letters

long, each letter one of 1A / 2A / 4A / 2B, read off the type of the sum of the two classes on that plane and CHECKED against the inner product of the two axes in the Griess form. Two concepts have a SIMILARITY, the exact rational $(g(a), g(b))$ in the Monster-invariant form, which is preserved by the group action. A pair in 2A position generates a 3-dimensional Norton-Sakuma algebra whose third axis is the axis of their PRODUCT: the Monster carrying out GLM composition, the Sakuma identity $a.b = (1/8)(a + b - a_{ab})$ verified in the register. A concept's plane gives an involution which two-colours the rest of the register. A concept's type-4 planes give coordinate frames, 24 orthogonal pairs each. And because the GLM-2 codec is linear, a concept splits into six FACETS — dimension, scale, tensor character, nominal kind, domain, context — each of which is again a point of Lambda and so again a word of ten Monster addresses, the six summing back to the carrier. That is enough to DECIDE an equation entirely inside the Monster, naming the facet that failed, in exact agreement with the GLM verdict over 7,140 library pairs; and to solve an analogy as lattice arithmetic. Sections 7 and 8.

(vi) THE ODD PART, AND THE WHOLE 196,884. The half of the Griess algebra

the second draft of this paper listed as missing — $V^- = 24 \times 4096 = 98,304$ — is built, with both of its multiplications, $V^+ \times V^- \rightarrow V^-$ and $V^- \times V^- \rightarrow V^+$. Its four structure constants are derived, not quoted, from two requirements: that the identity act as the identity, and that the Miyamoto involution of an axis be the extraspecial sign automorphism the

even part already produces. The system is over-determined and closes. The payoff is the eigenvalue ledger of a 2A axis on the WHOLE algebra,

$$1 / 96,256 / 4,371 / 96,256 \text{ at } 1 / 0 / 1/4 / 1/32,$$

totalling 196,884 — the classical numbers, none of which the even part can produce on its own. The products are commutative, Frobenius, and equivariant for Q; the fusion rules that involve the odd part, including $1/32 * 1/32 \rightarrow 1 + 0 + 1/4$, hold. Section 11.

(vii) THE SIGN OF AN AXIS IS SETTLED. a_{λ^+} and a_{λ^-} are both

idempotents of norm one, and the even part cannot tell them apart. Two things are done about it. A theta function on the Golay code — the same device that fixes signs in the Leech construction — gives a canonical GLOBAL sign convention, and the Sakuma identity is shown to FORCE $s(\lambda + \mu) = -s(\lambda) s(\mu)$, so the naive all-plus convention is incoherent and the canonical constant one is $s = -1$. Then the odd part resolves the question outright, not by convention: the two axes have DIFFERENT Miyamoto involutions, x_{λ} and $x_{\lambda} z$, which agree on the even part and differ on V^- . Sections 10 and 11.

(viii) DEPTH TEN IS A THEOREM, NOT A CONSTANT. The stack depth is derived

from the coordinate range of the data: the register's largest Leech-basis coordinate is 180, the least admissible parameters are offset 256 with depth 9, and the conventional offset 512 forces depth 10. The rebuild identity is proved for arbitrary admissible depth, deeper stacks are shown to append only zero planes, and the reasoning is measured to be depth-independent: two reasoners at different admissible depths return the same verdict on every pair. Section 9.

(ix) THE SIMILARITY BECOMES A METRIC. The invariant form is positive definite on the even part — it is twice a sum of squares in the (A, B) basis — so $d(a,b)^2 = (g(a) - g(b), g(a) - g(b))$ satisfies the triangle

inequality for free. It failed to be a metric only because the Griess vector is not injective: 660 concepts collapse onto 162 distinct vectors and 70 concepts carry no axis at all. Injectivity is restored by grading the embedding over the planes and letting a non-axis plane contribute a rank-one projector; the result separates the register, so nearest-neighbour and single-linkage clustering have guarantees rather than heuristics. Section 12.

(x) A BENCHMARK, NOT AN ANECDOTE. Four sections, reported as pass rate S:

the exhaustive pairwise sweep of all 217,470 pairs of the register (Monster verdict against GLM verdict), a corpus of 64 real physical laws, 224 deliberately corrupted mutants of them with the failing facet attributed to the mutation operator, and 40 dimensionless groups. Section 13.

What is NOT built is still stated as plainly as GLM-2 stated its own limits: the Monster itself is not generated — the extra generator that with N generates the group of order 8×10^{53} is not constructed — so what is here is a genuine subalgebra-and-subgroup picture, not the Monster. Section 14.

1. WHAT CAME BEFORE, AND WHY THE MONSTER

GLM-1 (./glm) put meaning in $(Z^7, +)$ and derived a 24-bit Golay carrier from it. Its central negative result is the mod-2 ceiling: a carrier whose composition law is XOR can only compare exponents modulo 2, so it cannot tell $E = m c^2$ from $E = m c^4$. Its central positive result is the fix: keep the exponents in a torsion-free group and make the carrier a DERIVED VIEW of them, never an input.

GLM-2 (./glm2) kept that discipline and raised everything: ten rational exponents plus scale, tensor rank and three parities; a Leech-lattice carrier with exact nearest-point repair; Co_0 constructed rather than quoted; an operator algebra; 660 concepts. It reached the Griess ledger

and stopped, because the algebra itself was out of reach at the time.

The reason to go on is not decoration. The Leech lattice has an enormous symmetry group, Co_0 of order 8.3×10^{18} , but Co_0 acts LINEARLY: it permutes and signs coordinates and nothing more. A linear group can move concepts around but it cannot multiply them. The Griess algebra is where the Leech lattice acquires a PRODUCT, and it is a product that is commutative, non-associative, and equipped with an invariant bilinear form — exactly the shape needed to ask "how far apart are these two meanings" and get an answer that is not a metric imposed from outside but an invariant of the structure. Moreover its idempotents of norm one, the $2A$ axes, are in bijection with the type-2 classes of $\Lambda / 2\Lambda$, of which there are 98,280 — so every concept whose carrier lands on a type-2 class IS an axis, and the relation between two concepts is a Norton-Sakuma type, one of a very short list, computable exactly.

That is the whole argument for using the Monster: it supplies, for free and canonically, a product on meanings and an invariant comparison of them. Everything below is the work of making that concrete.

2. THE RETAINED UBP / GLM LAYER

Nothing from GLM-2 is thrown away or reimplemented. `glm3_common` puts `../glm2` and `../glm` on the path and the GLM-3 reasoner holds a GLM-2 reasoner as its base, delegating:

<code>meaning(text)</code>	the exact rational meaning vector
<code>carrier(text)</code>	the derived Leech point
<code>audit(lhs, rhs)</code>	dimensional and full-meaning equation checking
<code>solve(target, srcs)</code>	the missing formula, by exact linear algebra
<code>convert(a, b)</code>	unit conversion through the decimal scale
<code>identify(text)</code>	every concept with that meaning
<code>repair(y)</code>	nearest-point decoding of a corrupted carrier
<code>list_concepts()</code>	the 660-concept register

The architecture invariant is inherited unchanged and re-checked here as claim C45: the meaning is the state, the carrier is encode(meaning), the carrier is never an input, and no verdict anywhere in the system is reached by reducing an exponent modulo 2. The Monster layer is strictly a READ of the carrier; it adds structure above the meaning and never feeds back into it. That is what makes it safe to add.

3. LAMBDA / 2LAMBDA: THE F₂ QUADRATIC SPACE THE MONSTER IS INDEXED BY

Module: glm3_leech2.py

GLM-2's integer model of the Leech lattice is scaled so that the minimal norm is 32 (it is the usual Lambda scaled by sqrt(8)). In that model, for lambda, mu in Lambda,

$$q(\text{lambda}) = (\text{lambda} \cdot \text{lambda}) / 16 \pmod{2}$$

$$B(\text{lambda}, \text{mu}) = (\text{lambda} \cdot \text{mu}) / 8 \pmod{2}$$

are well defined on Lambda / 2Lambda — q because the norm of lambda + 2mu exceeds that of lambda by 4(lambda.mu) + 4(mu.mu), and B because the

Gram matrix of the Leech basis is divisible by 8 off the diagonal. q is a quadratic form with polar form B; both are checked against the lattice directly (C5).

The type of a class is 0, 2, 3 or 4 according to the minimal norm of the vectors in it, which is 0, 32, 48 or 64 (that is, 2, 4, 6, 8 before the sqrt(8) scaling). These are the only possibilities, and the counts

$$1 + 98,280 + 8,386,560 + 8,292,375 = 16,777,216 = 2^{24}$$

come from the theta series: 196,560 / 2, 16,773,120 / 2 and

398,034,000 / 48, the last because a type-4 class contains 48 vectors forming a coordinate FRAME — 24 mutually orthogonal pairs (C8, C11).

The Witt decomposition computed in `witt_decomposition()` strips off hyperbolic planes one at a time by finding a singular vector and a partner with $B = 1$. It finds twelve planes and an empty anisotropic remainder, so the form is $O^+(24, 2)$, and the number of singular classes is the plus-type count $2^{(23)} + 2^{(11)} = 8,390,656$ (C6, C7).

Combining these two facts gives the $O(1)$ type test that the whole system runs on (C10):

```
type 0 iff the class is zero
type 3 iff q(class) = 1          (non-singular)
type 2 iff the class is in the 98,280-entry table
type 4 otherwise
```

because the non-singular classes number $2^{23} - 2^{11} = 8,386,560$, which is exactly the number of type-3 classes (C9). The alternative — running a Leech decoder — costs about 0.09 s per call; the table costs about 9 s once and every lookup after that is free.

4. $Q = 2^{(1+24)}$: THE COMPOSITION LAW

Module: `glm3_extraspecial.py`

Inside the Monster, the centraliser of a 2B involution is $2^{(1+24)}.Co_1$. Its normal subgroup Q is the extraspecial group of order 2^{25} associated with the quadratic space $(\Lambda/2\Lambda, q)$. We build it from q alone.

A cocycle is needed: a function f on pairs of classes with

$$\begin{aligned} f(u, u) &= q(u) \\ f(u, v) + f(v, u) &= B(u, v). \end{aligned}$$

Take a symplectic basis $\alpha_1.. \alpha_{12}$, $\beta_1.. \beta_{12}$ of singular vectors with $B(\alpha_i, \beta_j) = \delta_{ij}$ and all other pairings zero; such a basis exists precisely because the Witt decomposition found twelve hyperbolic planes, and it is found here by search (C13). Writing u in coordinates (a_u, b_u) in that basis, set

$$f(u, v) = \langle b_u, a_v \rangle \quad (\text{the standard } F_2 \text{ dot product}).$$

Then $f(u,u) = \langle b_u, a_u \rangle = q(u)$ and $f(u,v) + f(v,u) = \langle b_u, a_v \rangle + \langle b_v, a_u \rangle = B(u,v)$, both verified on random pairs. The group is the set of pairs (u, ϵ) with

$$(u, \epsilon) (v, \delta) = (u + v, \epsilon + \delta + f(u, v)),$$

and the relations z central, $z^2 = 1$, $x_u^2 = z^{q(u)}$ and $[x_u, x_v] = z^{B(u,v)}$ all hold (C14). Elements of order at most 2 number $2 * (\text{number of singular classes}) = 2^{24} + 2^{12}$, which identifies the plus type a second, independent way (C15).

The 4096-dimensional Schrodinger representation is built on F_2^{12} (the α -coordinates): $x_{\{a,b\}}$ sends the basis vector e_t to $(-1)^{\langle b,t \rangle} e_{t+a}$, and the centre to -1 . It is checked to be a homomorphism and faithful on samples (C16). Its dimension is the 4096 of the ledger: $24 \times 4096 = 98,304$ is the odd part of the Griess algebra (C17).

The bridge to GLM: composition of meanings ADDS carriers, and reduction mod

2 is additive, so the plane-0 classes add too, and therefore the group elements multiply, up to the central phase which records the cocycle:

$$x_{[a]} x_{[b]} = z^{f([a],[b])} x_{[a * b]}.$$

Claim C37 checks this on register concepts. The GLM product law is a group law in the Monster's 2B centraliser.

5. THE EVEN PART OF THE GRIESS ALGEBRA

Module: glm3_griess.py

The even part is

$$\begin{aligned} V_+ &= S^2(R^{24}) (+) \text{span}\{b_\lambda : \lambda \text{ a type-2 class}\} \\ &= 300 + 98,280 = 98,580 \text{ dimensional,} \end{aligned}$$

represented here as a pair (A, B) with A a symmetric 24 x 24 matrix of Fractions and B a sparse dictionary from type-2 class to Fraction. The product is

$$\begin{aligned} A_1 \cdot A_2 &= \alpha (A_1 A_2 + A_2 A_1) \\ A \cdot b_\lambda &= \beta (\lambda^T A \lambda) b_\lambda \\ b_\lambda \cdot b_\lambda &= \gamma P_\lambda \quad (P = \text{outer product}) \\ b_\lambda \cdot b_\mu &= \delta b_{\{\lambda + \mu\}} \quad \text{if } |\lambda + \mu| = 2 \\ &= 0 \quad \text{otherwise} \end{aligned}$$

and the invariant form is $(A_1, A_2) = \text{FA tr}(A_1 A_2)$, $(b, b) = \text{FB}$. The six constants are DERIVED, in this order:

- * the shape above is forced by equivariance under the monomial group $N = 2^{24} : \text{Co}_0$ acting on the two summands;
- * $\alpha = 1/2$ because the identity of the algebra must act as the identity on the matrix part, which makes $A_1 A_2$ the Jordan product;
- * $\beta = 1/4$ because the identity, which is the matrix $4I$ in this normalisation, must fix b_λ , and $\lambda^T (4I) \lambda = 4 * 32/8$;
- * $\text{FA} = \text{FB} = 2$ up to overall scale, from the Frobenius property $(x \cdot y, z) = (x, y \cdot z)$ tested across all four product shapes;
- * $\gamma = \delta = 1/4$ is then the unique choice making $a_\lambda = (1/8) P_\lambda + (1/2) b_\lambda$ idempotent of norm one with $\text{ad}(a_\lambda)$ having eigenvalue set $\{1, 0, 1/4, 1/32\}$.

Claims C19 to C22 verify each step, and the eigenspace dimensions $1 / 49,152 / 2,323 / 47,104$ are computed by counting, not quoted: the $49,152 = 2 * 24,576$ is the number of type-2 classes μ with $|\lambda.\mu| = 1$ in the standard scaling, and $2,323 = 300 + 2,023$ counts the matrix directions together with the classes orthogonal to λ , and so on. The total is 98,580 (C21).

A concept whose plane is a type-2 class therefore has, canonically, an IDEMPOTENT of the Griess algebra attached to it, obeying the Monster's fusion law. That is the sense in which the register lives in the Monster.

Norton-Sakuma (C25). Given two axes, the subalgebra they generate is computed by closure, and its dimension and the inner product identify the type:

$\lambda . \mu = 0 \rightarrow (a,b) = 0, \quad \dim 2, \text{ type } 2B$
 $|\lambda . \mu| = 1 \rightarrow (a,b) = 1/32, \quad \dim 5, \text{ type } 4A$
 $|\lambda . \mu| = 2 \rightarrow (a,b) = 1/8, \quad \dim 3, \text{ type } 2A$

and in the 2A case the Sakuma identity $a . b = (1/8)(a + b - a_{\rho})$ holds with a_{ρ} the MINUS-sign axis of the class $\lambda + \mu$. These are computed here, not looked up.

Miyamoto (C24). The Miyamoto involution of a_{λ} negates the $1/32$ eigenspace. On the even part it acts on b_{μ} by $(-1)^{|\lambda.\mu|}$, and $\lambda.\mu \bmod 2 = B(\lambda, \mu)$, so it coincides exactly with the extraspecial sign automorphism x_{λ} . Q acts on the even part by algebra automorphisms preserving the form. Restricted to the even part the Miyamoto map cannot separate the two axes a_{λ}^{+} and a_{λ}^{-}

—
they have the same $1/32$ eigenspace there. Section 11 separates them, on the odd part, where their Miyamoto involutions differ.

6. THE MULTI-MOG-CUBE

Module: glm3_mog.py

The Miracle Octad Generator is a 4×6 array of the 24 coordinates. There are two pictures of it as a cube, and they are two views of the same 24 cells.

VIEW ONE, the three $2 \times 2 \times 2$ cubes. The three 4×2 bricks each carry eight cells, addressed by three bits (row parity, row half, column) — an 8-bit cube. Each brick is an OCTAD, and the three are disjoint and cover everything, so they form a TRIO (C27). The six columns are tetrads and their pairwise unions are octads, so they form a SEXTET (C28). There are 759 octads, 3,795 trios and 1,771 sextets, all counted here (C31).

What lives on one cube? The archive material accompanying this project claims the Golay code restricted to a cube is the first-order Reed-Muller code $RM(1,3)$. That is FALSE and this paper refutes it (C29). The trace of the Golay code on a brick — the set of restrictions of all codewords — has 128 elements and is the EVEN-WEIGHT code $[8,7,2]$, not the 16-element $RM(1,3)$; the shortened code — codewords supported inside the brick — is $\{0, \text{the whole brick}\}$, of size 2. Both are computed exhaustively over all 4,096 codewords. The Reed-Muller structure people remember is a feature of a different construction, not of the Golay trace.

What DOES live on a cube is an affine geometry (C30). Fix a brick. Of the 759 octads, exactly 30 are disjoint from it; each cuts the complementary 16 cells in 8, and the 30 resulting subsets are hyperplanes of $AG(4,2)$ — they form 15 complementary pairs, and the associated partition labels are closed under symmetric difference and label the 16 cells bijectively by F_2^4 . The stabiliser is $2^4 : A_8 = AGL(4,2)$ of order 322,560, which matches the octad stabiliser order computed independently in GLM-1's M24 census. The octad intersection census $\{0:30, 2:448, 4:280, 8:1\}$ is the standard one and is computed here.

VIEW TWO, the digit-plane stack. Write a Leech point in the 24 AMBIENT

coordinates and stack its binary digit planes. Then plane 0 is constant (all coordinates share a parity), plane 1 is a GOLAY CODEWORD, and the planes above satisfy a mod-8 condition on the coordinate sum (C32). That is not an observation about digits: it is a restatement of the three Leech congruences. The MOG cube picture and the digit-stack picture are the same

object. Every codeword aligned to the MOG casts a HEXACODE shadow — the six column-classes in $GF(4)$ — and that too is verified across all 4,096 codewords (C26).

The system uses the second view in the LEECH BASIS rather than the ambient one, because it is there that each plane is a class of $\Lambda / 2\Lambda$, hence a Monster address. That is the next section.

7. THE BRIDGE: A CONCEPT AS A WORD OF TEN MONSTER ADDRESSES

Module: glm3_reasoner.py

THE PROBLEM. Reduce a concept's carrier mod 2 and you get one class. Across the whole register that map has an image of size NINE (C34). The reason is structural: the GLM-2 codec writes rational exponents into integer slots by clearing a denominator, so nearly every slot is even, and reduction kills them. This is the mod-2 ceiling of GLM-1, met again.

THE FIX. Do not reduce; expand. Offset the Leech-basis coordinate vector by 2^9 in every slot (so that negatives have binary expansions) and read off ten digit planes:

$$\text{plane } k \text{ of } x = \sum \text{ over } i \text{ of } \text{bit } k \text{ of } (x_i + 512) \text{ times } 2^i.$$

Each plane is a 24-bit vector, hence a class of $\Lambda / 2\Lambda$, hence a Monster address; and

`class_stack_rebuild(class_stack(x)) == x`

for every concept in the register (C35). Nothing is lost. A concept is a WORD of ten Monster addresses, and everything the Monster can say about one

address, it can now say ten times about a concept.

WHAT THE MONSTER THEN SAYS.

- * The TYPE WORD, ten digits from $\{0,2,3,4\}$. Over the register the planes distribute $\{0: 2058, 2: 1212, 3: 1175, 4: 2155\}$, and 590 of the 660 concepts have at least one type-2 plane, i.e. at least one Majorana axis (C36).
- * The GRIESS VECTOR $g(c) = \text{sum over axis planes of } 2^{-k} a_{\{d_k\}}$, an exact element of the 98,580-dimensional algebra, weighted so that the low planes — the fine structure of the meaning — dominate.
- * The SIMILARITY $(g(a), g(b))$, an exact rational, invariant under the group action (C41). Energy and speed score $5/8$, energy and force $83/256$, energy and mass $3/512$.
- * The RELATION WORD. On each plane, add the two classes; the type of the sum names the Monster class of the product of the two 2A involutions,

sum type 0 $\rightarrow$ 1A, 2 $\rightarrow$ 2A, 3 $\rightarrow$ 4A, 4 $\rightarrow$ 2B,

and where both planes are themselves axes the predicted inner product 1, $1/8$, $1/32$ or 0 is CHECKED in the algebra (C38). Across the first forty concepts the plane-0 census is $\{1A: 264, 2A: 373, 2B: 65, 4A: 78\}$.

- * The TRIANGLE. If two concepts are in 2A position on plane 0 then the third axis of their Norton-Sakuma algebra is the axis of their PRODUCT concept, and the Sakuma identity holds (C40). This is the sharpest form of the thesis: the Monster's 3-dimensional 2A algebra is performing GLM

composition.

- * The INVOLUTION. A plane of a concept gives x_u , which fixes the axes of concepts with $B(u,v) = 0$ and negates the rest: a canonical two-colouring of the register by any chosen concept (C42). Plane 0 is degenerate here and colours everything the same, for the reason given above — the codec's lowest digit plane is nearly constant across the register — while plane 2 of "energy" splits the 660 concepts 387 / 273. This is a concrete measurement of how much information each plane carries, and it is why the system uses the whole stack rather than any single plane.
- * The FRAME. A type-4 plane resolves into 48 lattice vectors in 24 orthogonal pairs, a coordinate frame — a 24-fold orthogonal decomposition attached to that plane of the concept (C43).

FACETS, AND REASONING THAT NEVER LEAVES THE MONSTER. The stack is

faithful, so it is not a commentary on the carrier: it IS the carrier, rewritten. That makes it possible to do the reasoning up here, and the key is that the codec is LINEAR. glm2_codec writes a meaning into 24 named integer slots — ten rational exponents, the decimal scale, rank, the three P/T/C fields, a nominal kind, a domain, seven free context slots — and then applies the Leech basis. Zeroing a group of slots is therefore a lattice projection. Each of the six FACETS

dimension	slots 0-9	scale	slot 10	tensor	slots 11-14
kind	slot 15	domain	slot 16	context	slots 17-23

sends a concept to another point of Lambda, hence to another word of ten Monster addresses with its own types and its own axes; and because the facets partition the slots, the six facet points sum back exactly to the carrier (C47). A concept is not one Monster word but a structured family of seven of them.

- * DECIDING AN EQUATION. GLM-2 admissibility is a conjunction of facet questions: same dimension, same scale, same tensor character, and no

clash of nominal kind. Each becomes a word comparison, so the verdict is reached without ever reading a meaning vector — and the failing facet is NAMED, because it is the one whose word differed. Over all 7,140 pairs from 120 library concepts, plus a batch of parsed expressions, the Monster verdict and the GLM verdict never disagree (C48). Nominal kind needs the "clash only if both are labelled" rule, and that too is in-Monster: a facet is unlabelled exactly when its word is the word of the origin.

The report also states what PLANE 0 OF THE FULL CARRIER alone would have

said. Plane 0 is a single reduction mod 2; on $E = m c^2$ vs $E = m c^4$ it says "agree", while the dimension facet differs on planes 3, 4 and 5. That is the mod-2 ceiling of GLM-1, exhibited from inside the Monster, and it is why the system reasons with the whole stack.

The facets also let the Monster see MORE than the verdict uses. The domain facet distinguishes force from mass*acceleration, which GLM rightly calls admissible; monster_check reports both, so "admissible, but declared in different namespaces" is now sayable.

* ANALOGY. $\text{analogy}(a, b, c)$ answers "a is to b as c is to what?". Composition of meanings adds carriers, so the answer's carrier is $x_b - x_a + x_c$, which is again a point of Lambda and so has its own word of ten addresses and its own Griess vector. The register is ranked against it by number of agreeing planes first and Griess similarity second. $\text{mass} : \text{force} :: \text{time} : ?$ returns four_velocity on 9 of 10 planes, then alfven_speed, ion_sound_speed and speed — the speed family, which is right, since force/mass is an acceleration and acceleration x time is a speed (C49). Where a concept sits exactly on the target point it is named as the exact answer, and $a : a :: c : ?$ returns c.

8. THE COMPANION IMPLEMENTATION

```
python3 glm3_reasoner.py          the twelve-section demonstration
python3 glm3_reasoner.py address energy
python3 glm3_reasoner.py stack energy
python3 glm3_reasoner.py relation energy torque
python3 glm3_reasoner.py similar energy
python3 glm3_reasoner.py triangle
python3 glm3_reasoner.py fusion energy
python3 glm3_reasoner.py orbit energy
python3 glm3_reasoner.py frame energy
python3 glm3_reasoner.py mog energy
python3 glm3_reasoner.py census
python3 glm3_reasoner.py facets energy
python3 glm3_reasoner.py check "mass*speed^2" "mass*speed^4"
python3 glm3_reasoner.py analogy mass force time
python3 glm3_reasoner.py audit energy "mass*speed^2"
python3 glm3_reasoner.py solve energy mass speed
```

The new commands, from the five sections that follow:

```
python3 glm3_reasoner.py distance energy work
python3 glm3_reasoner.py nearest energy
python3 glm3_reasoner.py cluster 0.05
python3 glm3_reasoner.py ledger
python3 glm3_reasoner.py odd energy
```

Each module also runs standalone and audits itself:

```
python3 glm3_leech2.py      python3 glm3_griess.py
python3 glm3_extraspecial.py python3 glm3_mog.py
python3 glm3_sign.py        python3 glm3_odd.py
python3 glm3_metric.py      python3 glm3_bench.py
```

9. THE DEPTH IS DERIVED, NOT CHOSEN

Module: glm3_leech2.py (coordinate_range, derive_stack_parameters,
class_stack, class_stack_rebuild, depth_report)

Ten was a magic number: the coordinates happened to fit in ten bits after the offset 2^9 . It is now a measurement, and the faithfulness of the stack is a proposition about arbitrary depth.

PROPOSITION D1. Let the data have Leech-BASIS coordinates bounded in absolute value by R . Let the offset O satisfy $O \geq R$ and let the depth D satisfy $2^D > O + R$. Then for every such point x , with coordinates u_i ,

$$0 \leq u_i + O < 2^D,$$

so each shifted coordinate has a D -digit binary expansion, plane k of the stack is the 24-bit mask of the k -th digits, and

$$\text{rebuild_O}(\text{stack}_{\{D,O\}}(x)) = x.$$

Proof: reading the digits and reassembling $\sum_k 2^k d_k - O$ is the identity on $[0, 2^D)$. Nothing in it mentions ten. The Lean companion RequestProject/GLM3.lean carries this as a machine-checked theorem about binary digit expansions at arbitrary depth.

THE MEASUREMENT. `coordinate_range` over the 660 concepts is 180. The least

admissible pair is therefore offset 256, depth 9; the module's conventional offset 512 -- kept because a power-of-two offset makes the shift a shift of digit planes -- forces depth 10. So `STACK_DEPTH = 10` is now DERIVED from the register, and ``depth_report`` recomputes it (C50).

ABOVE THE THRESHOLD, DEPTH DOES NOT MATTER. At a fixed offset, planes at

or above the least admissible depth are identically zero, and the planes below it do not move (C51). A zero plane is the class of the origin, carries no axis, and contributes nothing to any word, any Griess vector or any distance. The reasoner takes depth and offset as constructor

arguments, and two reasoners built at different admissible parameters -- (256, 9) and (512, 12) -- return the SAME verdict on every pair tried, and the same type word up to trailing zeros (C52, C53). Depth is a presentation, not a parameter of the mathematics.

10. THE SIGN OF AN AXIS: A COHERENT GLOBAL CONVENTION

Module: glm3_sign.py

Every type-2 class carries TWO axes, $a_{\lambda^{\{+-\}}} = (1/8) P_{\lambda} + (1/2) b_{\lambda}$, both idempotent, both of norm one, with the same spectrum. The reasoner has to choose one for each class, and the choice must be COHERENT: the Sakuma identity relates the three axes of a 2A triangle, so the choices on λ , μ and $\lambda + \mu$ cannot be made independently.

THE THETA FUNCTION. The device that fixes signs in the Leech and Monster constructions is a quadratic function on the code with a prescribed polarisation. On the Golay code, $\theta(C) = |C|/4 \pmod{2}$ is such a function: weights are multiples of four, and

$$\theta(C + D) = \theta(C) + \theta(D) + |C \text{ and } D| / 2 \pmod{2},$$

the correction term being the polar form. θ is not linear (C54). It is exactly the Leech form seen from the code: for the Leech point $2 * 1_C$, the class quadratic form q of section 3 satisfies

$$q([2 * 1_C]) = \theta(C)$$

for all 4,096 codewords -- verified exhaustively (Proposition S2). From the Gram matrix the module also builds the bimultiplicative lattice cocycle ϵ with $\epsilon(x,y) \epsilon(y,x) = (-1)^{\{x,y\}}$ and $\epsilon(x,x) = (-1)^{\{x,x\}/2}$, which is the same construction one level down.

WHAT THE ALGEBRA FORCES (Proposition S1, computed). Write the convention as

a sign $s(\lambda)$ in $\{+1, -1\}$ per class. Evaluating the Sakuma identity on a 2A triangle $\{\lambda, \mu, \lambda + \mu\}$ in the algebra gives

$$s(\lambda + \mu) = -s(\lambda) s(\mu),$$

so the all-plus convention is INCOHERENT -- it fails on every triangle -- while the constant convention $s = -1$ is coherent. That is why

CANONICAL_SIGN = -1 and why the reasoner's triangle report no longer needs the ad-hoc minus sign it used to carry (C55).

HOW MANY COHERENT CONVENTIONS ARE THERE? Writing $s = -(-1)^t$, the condition

becomes $t(\lambda) + t(\mu) + t(\lambda + \mu) = 0$: t is additive along triangles. Solving that F_2 system on closed subsystems -- all type-2 classes inside a subspace W of dimension 16, 18, 20, 22 (408, 1,512, 6,120, 24,552 classes) -- gives nullity exactly 24 every time, and the conventions $t = B(w, -)$ already supply $\dim W$ independent solutions. So the coherent conventions form a single orbit of $2^{24} = |Q/Z|$ solutions, permuted simply transitively by Q , and the canonical one is the unique CONSTANT member of that orbit. The evidence is the constancy of the nullity across four subsystem sizes, and it is labelled as evidence.

This settles the sign as a convention. Section 11 settles it as mathematics.

11. THE ODD PART, AND THE WHOLE 196,884

Module: glm3_odd.py

$V^- = R^{24}(x) R^{4096}$, basis $e_i(x) f_m$, the second factor the Schrodinger representation of Q built in section 4. For a type-2 class $q(\lambda) = 0$,

so $X_{\lambda} = \rho(x_{\lambda})$ is an involution and is symmetric -- which is what makes the form on V^- invariant.

THE TWO PRODUCTS. N -equivariance leaves exactly one shape for the action of the even part:

$$\begin{aligned} A |> (x(x)s) &= [c_1 A x + c_2 \text{tr}(A) x] (x)s \\ b_{\lambda} |> (x(x)s) &= [c_3 (\lambda \cdot x) \lambda + c_4 x] (x) X_{\lambda} s \end{aligned}$$

because the commutant of the stabiliser of λ in the 24-space is spanned by the identity and the projector onto λ . Four constants; two requirements pin them down.

(i) the identity of V^+ must act as the identity: $c_1 + 24 c_2 = 1$;

(ii) MIYAMOTO. The involution that is $+1$ on the $1, 0$ and $1/4$ eigenspaces of an axis and -1 on its $1/32$ eigenspace must be an automorphism, and on V^+ section 5 already identifies it as x_{λ} , which acts on V^- as $1(x) X_{\lambda}$. So on V^- the $1/32$ eigenspace must be exactly one eigenspace of X_{λ} . Splitting x into the λ direction and its complement and s into the ± 1 eigenspaces of X_{λ} gives four blocks with eigenvalues $c_2/2 \pm c_4/2$ (perp) and $(c_1+c_2)/2 \pm (4c_3+c_4)/2$ (along); requiring $1/32$ on one X_{λ} eigenspace and $\{0, 1/4\}$ on the other, with the $1/4$ forced onto the 2,048-dimensional along block by the count $4,371 - 2,323 = 2,048$, gives four equations for three unknown combinations. One of them is a consistency check, and it closes.

Hence, DERIVED,

$$c_1 = 1/4, \quad c_2 = 1/32, \quad c_3 = 1/16, \quad c_4 = -1/32.$$

The Miyamoto requirement is doing real work: the constants that reproduce the SPECTRUM but pair the blocks the other way round ($c_3 = 3/64, c_4 = 1/32$)

give exactly the same eigenvalues and dimensions, are still commutative and still Frobenius, and FAIL the fusion rule $1/32 * 1/32 \rightarrow 1 + 0 + 1/4$. The grading is the content, and the failure is reproducible by changing two constants in the module.

The second product is then not chosen at all. The form on V_+ is nondegenerate -- section 12 shows it is positive definite -- so the Frobenius requirement

$$(u \cdot v, w)_+ = (w |> u, v)_- \quad \text{for all } w \text{ in } V_+$$

DEFINES the map $V_- (x) V_- \rightarrow V_+$ uniquely; reading it off on the basis gives a closed form whose b_μ coefficient is nonzero only when $\alpha_\mu = m + n$,

a coset of the 4,096-element kernel, of which about two dozen classes are of type 2. That is what makes a product of two odd vectors computable one coordinate at a time.

THE LEDGER (C56). The block eigenvalues give 47,104 at 0, 2,048 at $1/4$ and 49,152 at $1/32$ on V_- ; with the even part's $1 / 49,152 / 2,323 / 47,104$,

eigenvalue	1	0	$1/4$	$1/32$	
dimension	1	96,256	4,371	96,256	total 196,884 ,

the classical eigenspace dimensions of a $2A$ axis of the Monster -- computed here, and reachable only with the odd part present.

WHAT IS CHECKED (C57, C58). Commutativity; the Frobenius identity as a computation; Q -equivariance of both products; every block eigenvector explicitly, against the block prediction; the fusion rules that involve the odd part, including $1/32 * 1/32 \rightarrow 1 + 0 + 1/4$ tested in V_+ with the even part's own eigen-filter; and the Miyamoto statement itself.

THE SIGN, RESOLVED. On V_- the eigenvalue of $\text{ad}(a_\lambda^\epsilon)$ is $1/32$ exactly on the X_λ -eigenspace $\sigma = -\epsilon$. Therefore

$$\tau(a_{\lambda}^-) = x_{\lambda} , \quad \tau(a_{\lambda}^+) = x_{\lambda} z ,$$

two DIFFERENT automorphisms of the 196,884-dimensional algebra which restrict to the same automorphism of the even part. On the single vector $\lambda(x)$ s with $X_{\lambda} s = s$ the canonical axis acts by $1/32$ and the other by $1/4$: one application of the algebra separates them. Section 10 fixed the sign by convention; this fixes it by observation, which is what the earlier draft said was missing.

12. FROM A SIMILARITY TO A METRIC

Module: glm3_metric.py

POSITIVE DEFINITENESS IS FREE. In the (A, B) basis the invariant form is

$$\begin{aligned} (x, x) &= 2 \operatorname{tr}(A^2) + 2 \sum_{\lambda} B_{\lambda}^2 \\ &= 2 \sum_{\{i \leq j\}} (2 - [i=j]) A_{ij}^2 + 2 \sum B_{\lambda}^2 , \end{aligned}$$

a positive combination of squares of the coordinates, because the two form constants derived in section 5 are both $+2$. So the form is positive definite on the even part, and

$$d(a, b)^2 = (g(a) - g(b), g(a) - g(b))$$

is a pseudometric: symmetry and the triangle inequality come for free from the Cauchy-Schwarz inequality of a positive semidefinite form, and are also checked as exact rational arithmetic on sampled triples (C59).

WHY IT WAS ONLY A PSEUDOMETRIC. g is not injective. Measured on the register: the 660 concepts have 162 distinct Griess vectors, 70 of them carry no axis on any plane and so collapse to the zero vector, and the largest fibre has 90 members. Two fixes are implemented, and both are honest metrics.

QUOTIENT. Identify concepts with the same Griess vector. d descends to a genuine metric on the 162 classes. This is exactly the statement that a pseudometric induces a metric on the quotient by distance zero, and ``quotient_report`` computes the fibres.

RESTORE INJECTIVITY (Proposition M3). Let every plane contribute, not only the axis planes. Define, for a class c ,

$$\begin{aligned} v(c) &= \text{the canonical axis of } c, & \text{if } c \text{ is of type 2,} \\ v(c) &= \eta * (\text{rank-one projector onto the line of the 0/1 coordinate} \\ &\quad \text{representative of } c), & \text{otherwise, and } v(0) = 0, \end{aligned}$$

and grade over the stack,

$$\begin{aligned} G(x) &= (2^{-k} v(d_k(x)))_{\{k < \text{depth}\}} \text{ in } (V+)^{\text{depth}}, \\ d(x, y)^2 &= \sum_k 4^{-k} (v(d_k x) - v(d_k y), v(d_k x) - v(d_k y)). \end{aligned}$$

v is injective on classes and vanishes only at the zero class, so G is injective on stacks; the stack is faithful (section 9), so G is injective on carriers, and d is a metric on carriers (Corollary M4). Since the plane distance depends on the two classes alone it is computed in $O(24)$ and cached, which is what makes a 660×660 sweep a few seconds' work.

MEASURED ON THE REGISTER. The graded metric separates the register: every

pair of concepts with different carriers is at positive distance, and distance zero occurs exactly for concepts sharing a carrier (energy and work, for instance). The closest distinct pair is (four_momentum, impulse) at 0.003413 and the farthest is (binding_energy, christoffel_symbol) at 1.947967 (C60).

WHAT IT BUYS. Nearest-neighbour queries and single-linkage clustering with guarantees: the partition at a threshold depends on the data and the threshold alone, not on visit order or seeding, because the relation being closed is a metric ball. ``reasoner.distance``, ``reasoner.nearest`` and

`reasoner.cluster` expose it, and the old form-based `neighbours` is kept and clearly labelled as a ranking, not a distance.

13. THE BENCHMARK

Module: glm3_bench.py python3 glm3_bench.py [laws|mutants|numbers|sweep]

One worked example is an anecdote. This is the evaluation. Four sections, each with an unambiguous right answer, reported as pass rates.

A. EXHAUSTIVE PAIRWISE SWEEP (C61). All 217,470 pairs of the 660-concept

register, the in-Monster verdict against the GLM verdict: 217,470 agreements, zero disagreements, 9,676 admissible and 207,794 rejected. The Monster verdict is evaluated in an unrolled form -- facet words cached per concept, a pair being four tuple comparisons -- which makes the sweep seconds rather than an hour; that the unrolled form is the same computation is not assumed but cross-checked against `monster\_check` on 2,000 pairs.

B. A CORPUS OF REAL LAWS (C62). 64 laws -- Newton's second law, Coulomb,

gravitation, Ohm, the ideal gas law, Planck, Stefan-Boltzmann, Bernoulli, drag, Lorentz force, Poynting, Larmor, de Broglie, Compton, Rydberg, Josephson, Hall, Arrhenius, Beer-Lambert, Fick, Fourier, Darcy, Bragg, Boltzmann entropy, the Friedmann equations, and individual Navier-Stokes and Maxwell terms among them. Every one comes out admissible, and the Monster agrees with GLM on every one. Vector laws are written with a vector factor (force = spring_constant * position, not * length) because the register distinguishes rank and parity; that is a property of the corpus, stated, not a fudge.

C. MUTANTS, WITH FACET ATTRIBUTION (C63). Every law is corrupted four

ways

-- perturb one exponent, swap two quantities, change a rank, change a decimal scale -- giving 224 genuine corruptions after discarding the mutations that leave the meaning unchanged. All 224 are caught: 218 by verdict and 6 refused by the parser; the false-negative rate is zero. The facet report earns its keep here, because it names which facet caught each mutant:

exponent -> dimension 52, dimension+tensor 9

swap -> dimension 26, dimension+tensor 3

rank -> tensor 64

scale -> scale 64

exactly the attribution the construction predicts.

D. DIMENSIONLESS NUMBERS (C64). 40 groups -- Reynolds, Mach, Prandtl, Nusselt, Froude, Weber, Peclet, Strouhal, Knudsen, Biot, Rossby, Ekman, Schmidt, Lewis, Grashof, Rayleigh, Bond, capillary and more, several in more than one equivalent form. 38 come out exactly dimensionless. The two exceptions are reported rather than hidden: written in the textbook way with an angular velocity, Rossby and Ekman carry A^{-1} , a power of the radian, because GLM-2 promotes plane angle to a dimension of its own; written with a frequency they are exactly dimensionless. This section stresses the codec's fractional-exponent handling in a way pairwise comparison does not.

14. WHAT IS NOT BUILT

Stated plainly, in the manner of GLM-2 section 8. Four of the five items this section used to list have now been built; what they say here is what is still true after building them.

(a) THE ODD PART -- BUILT (section 11). $V = 24 \times 4096 = 98,304$ with both

multiplications, derived constants, the 196,884 ledger, Q-equivariance and the fusion rules. What is NOT claimed: the products are verified on explicit block eigenvectors and on sampled vectors rather than by a symbolic proof over all 196,884 dimensions, and no identity beyond commutativity, Frobenius, equivariance and the fusion law is asserted.

- (b) THE MONSTER ITSELF -- STILL NOT BUILT. Only N-side symmetries are constructed: Q by generators and relations, and its action by automorphisms. The extra generator that with N generates the Monster is not built, so no claim in this paper is a claim about the full group of order 8×10^{53} . What IS constructed is a genuine subgroup acting genuinely on a genuine 196,884-dimensional algebra.
- (c) SIGN OF AN AXIS -- SETTLED (sections 10 and 11). There is a canonical coherent global convention, forced by the Sakuma identity and matching the Golay theta construction, and the odd part distinguishes the two axes outright by their Miyamoto involutions. What is NOT claimed: the count of coherent conventions is 2^{24} on the evidence of four closed subsystems, not of an elimination over all 98,280 unknowns.
- (d) DEPTH TEN -- DERIVED (section 9). Depth and offset are parameters, derived from the coordinate range, with the rebuild identity proved for arbitrary admissible depth and depth-independence of the verdicts measured. What is NOT claimed: depth-independence is measured over the register's pairs, not proved for all possible inputs.
- (e) THE SIMILARITY -- NOW A METRIC (section 12). Positive definiteness gives the triangle inequality; the graded embedding gives injectivity; nearest neighbours and clustering are well defined. What is NOT claimed: that the metric is canonical. The grading weights 2^{-k} and the weight η on a non-axis plane are choices, stated as choices; any positive values give a metric, and no verdict depends on which.

-
1. The meaning is the state. The carrier is encode(meaning). The Monster layer READS the carrier and never writes to it. (C45)
 2. No verdict is reached by reducing an exponent mod 2. Reduction mod 2 appears only where the mathematics is genuinely over F_2 — the class group $\Lambda/2\Lambda$ — and there it is never a verdict, only an index.
 3. `class_stack_rebuild o class_stack` = identity on every carrier. (C35)
 4. The $O(1)$ type test agrees with the decoder. (C10)
 5. The four structure constants stay derived: if any is changed, C20-C23 fail immediately, and the four constants of the odd part are derived the same way: change one and C56-C58 fail.
 6. All arithmetic above the lattice is in Fraction, never float.
 7. The stack depth stays derived from the coordinate range, and the reasoning stays depth-independent above the threshold. (C50-C53)
 8. The axis sign convention stays coherent — a change to it must keep the Sakuma identity holding with no ad-hoc sign. (C55)
 9. The distance stays a metric on the register: positive definiteness of the form and injectivity of the graded embedding. (C59, C60)
 10. The benchmark stays at full pass rates in all four sections, and any new law, mutant or dimensionless group is added to the corpus rather than tested by hand. (C61-C64)

=====

=====

""""

```
from __future__ import annotations
```

```
import json
```

```
import os
```

```
import random
```

```
import sys
```

```
import time
```

```
from fractions import Fraction as F
```

```
from typing import Callable, Dict, List, Optional, Sequence, Tuple
```

```
import glm3_common # noqa: F401 (path shim: puts ../glm2 and ../glm on sys.p
ath)
```

```
import glm3_extraspecial as XS
import glm3_griess as GR
import glm3_leech2 as L2
import glm3_mog as MG
from glm3_reasoner import REASONER, RELATION_BY_PRODUCT_TYPE
```

```
import glm2_lattice as LAT
from glm2_codec import compose, encode, repair
from glm2_library import library_audit
```

```
# =====
# CLAIM MACHINERY
# =====
```

```
Verifier = Callable[[], Tuple[bool, Dict[str, object]]]
```

```
CLAIMS: List[Tuple[str, str, Verifier, bool]] = []
```

```
def claim(cid: str, statement: str, heavy: bool = False):
    """Register a numbered claim together with the function that verifies it."""
    def wrap(fn: Verifier) -> Verifier:
        CLAIMS.append((cid, statement, fn, heavy))
        return fn
    return wrap
```

```
def _rng() -> random.Random:
    return random.Random(20260816)
```

```
#
```

```
# §2. THE RETAINED UBP / GLM LAYER
```

```
#
```

```
@claim("C1", "the GLM-2 register is carried into GLM-3 unchanged: 660 concepts, "
```

```
    "every defining relation exact, every concept encodable")
```

```
def c1():
```

```
    rep = library_audit()
```

```
    ok = (rep["relations_ok"] == rep["relations_checked"]
```

```
        and rep["tensor_relations_ok"] == rep["tensor_relations_checked"]
```

```
        and rep["all_encodable"] and rep["concepts"] >= 660)
```

```
    return ok, {k: rep[k] for k in ("concepts", "domains", "distinct_meanings",
```

```
        "relations_checked", "relations_ok",
```

```
        "tensor_relations_checked",
```

```
        "tensor_relations_ok", "all_encodable")}
```

```
@claim("C2", "dimensional reasoning still works:  $E = m c^2$  is admissible, "
```

```
    " $E = m c^4$  is rejected, and the missing formula is solved for")
```

```
def c2():
```

```
    good = REASONER.audit("energy", "mass*speed^2")
```

```
    bad = REASONER.audit("energy", "mass*speed^4")
```

```
    force = REASONER.audit("force", "mass*acceleration")
```

```
    solved = str(REASONER.solve("energy", ["mass", "speed"]))
```

```
    conv = REASONER.convert("kilometre", "centimetre")
```

```
    checks = {
```

```
        "E = m c^2 admissible": bool(good.admissible),
```

```
        "E = m c^4 rejected": not bool(bad.admissible),
```

```
        "F = m a admissible": bool(force.admissible),
```

```
        "solve energy from mass, speed": solved,
```

```
        "solve is right": solved == "energy = mass * speed^2",
```

```
        "convert km to cm": conv,
```

```

}
ok = (checks["E = m c^2 admissible"] and checks["E = m c^4 rejected"]
      and checks["F = m a admissible"] and checks["solve is right"]
      and conv is not None)
return ok, checks

@claim("C3", "the carrier is still a derived view of the meaning, still "
        "injective on meanings, and still exactly repairable")
def c3():
    names = REASONER.list_concepts()
    seen: Dict[Tuple[int, ...], str] = {}
    injective = True
    for n in names:
        x = tuple(REASONER.carrier(n))
        if x in seen and REASONER.meaning(seen[x]) != REASONER.meaning(n):
            injective = False
        seen.setdefault(x, n)
    x = list(REASONER.carrier("energy"))
    y = list(x)
    y[0] += 1
    y[7] -= 1
    res = repair(y)
    a, b = REASONER.meaning("force"), REASONER.meaning("length")
    composes = tuple(encode(a + b)) == tuple(compose(encode(a), encode(b)))
    ok = (injective and res.within_radius
          and res.meaning == REASONER.meaning("energy") and composes)
    return ok, {
        "concepts": len(names),
        "distinct carriers": len(seen),
        "distinct meanings give distinct carriers": injective,
        "repair of a squared-error-2 corruption": str(res),
        "repair recovers the meaning exactly":
            res.meaning == REASONER.meaning("energy"),
        "carrier of a product = compose of carriers": composes,
    }

```

```
# =====
```

```
# §3. LAMBDA / 2LAMBDA
```

```
# =====
```

```
@claim("C4", "the 24 Leech-basis vectors reduce to a basis of Lambda/2Lambd  
a, "
```

```
    "so the class group really is  $F_2^{24}$ ")
```

```
def c4():
```

```
    classes = [L2.class_of(row) for row in LAT.LEECH_BASIS]
```

```
    span = {0}
```

```
    for c in classes:
```

```
        span |= {s ^ c for s in span}
```

```
    ok = len(span) == L2.N_CLASSES and len(set(classes)) == 24
```

```
    return ok, {"basis classes distinct": len(set(classes)) == 24,
```

```
               "span size": len(span), "expected": L2.N_CLASSES}
```

```
@claim("C5", "q(x) = (x.x)/16 mod 2 and B(x,y) = (x.y)/8 mod 2 are well "
```

```
    "defined on Lambda/2Lambda and q is a quadratic form with polar B")
```

```
def c5():
```

```
    rep = L2.leech2_audit(full=False)
```

```
    rng = _rng()
```

```
    quad = True
```

```
    for _ in range(300):
```

```
        u = rng.randrange(L2.N_CLASSES)
```

```
        v = rng.randrange(L2.N_CLASSES)
```

```
        if L2.q_form(u ^ v) != (L2.q_form(u) + L2.q_form(v)  
                                + L2.b_form(u, v)) % 2:
```

```
            quad = False
```

```
            break
```

```
    bilinear = all(  
        L2.b_form(u ^ v, w) == (L2.b_form(u, w) + L2.b_form(v, w)) % 2
```

```

    for u, v, w in ((rng.randrange(L2.N_CLASSES),
                        rng.randrange(L2.N_CLASSES),
                        rng.randrange(L2.N_CLASSES)) for _ in range(300)))
ok = (rep["q_matches_lattice"] and rep["b_matches_lattice"]
      and rep["q_well_defined"] and quad and bilinear)
return ok, {
    "q agrees with the lattice norm": rep["q_matches_lattice"],
    "B agrees with the lattice inner product": rep["b_matches_lattice"],
    "q well defined modulo 2Lambd": rep["q_well_defined"],
    "q(u+v) = q(u) + q(v) + B(u,v)": quad,
    "B is bilinear": bilinear,
}

```

@claim("C6", "the Witt decomposition of q is twelve hyperbolic planes and an "
 "empty anisotropic part, so the form is of PLUS type")

```

def c6():
    w = L2.witt_decomposition()
    ok = (w["planes"] == 12 and w["anisotropic_planes"] == 0
          and w["plus_type"] and L2.form_is_plus_type())
    return ok, {k: w[k] for k in ("planes", "anisotropic_planes", "plus_type",
                                "singular_count")}

```

@claim("C7", "the singular classes number $2^{23} + 2^{11} = 8,390,656$ and the "
 "non-singular ones $2^{23} - 2^{11} = 8,386,560$ ")

```

def c7():
    sing = L2.singular_class_count()
    ok = (sing == (1 << 23) + (1 << 11)
          and L2.N_CLASSES - sing == (1 << 23) - (1 << 11))
    return ok, {"singular": sing, "2^23 + 2^11": (1 << 23) + (1 << 11),
               "non-singular": L2.N_CLASSES - sing,
               "2^23 - 2^11": (1 << 23) - (1 << 11)}

```

@claim("C8", "the class census $1 + 98,280 + 8,386,560 + 8,292,375 = 2^{24}$ ")

"closes, derived from the theta series of the lattice")

```
def c8():
```

```
    cen = L2.type_census()
```

```
    ok = cen["closes"] and cen["total"] == L2.N_CLASSES
```

```
    return ok, {k: cen[k] for k in ("theta", "type2_vectors", "type3_vectors",  
                                   "type4_vectors", "type2_classes",  
                                   "type3_classes", "type4_classes",  
                                   "total", "closes")}
```

```
@claim("C9", "the type-3 classes are exactly the non-singular ones: "
```

```
    "8,386,560 = 223 - 211")
```

```
def c9():
```

```
    cen = L2.type_census()
```

```
    ok = (cen["type3_classes"] == cen["nonsingular"]
```

```
        and cen["matches_plus_type"])
```

```
    return ok, {"type-3 classes": cen["type3_classes"],  
               "non-singular classes": cen["nonsingular"],  
               "agree": cen["type3_classes"] == cen["nonsingular"]}
```

```
@claim("C10", "the O(1) type test (zero / q=1 / table / else) agrees with the "
```

```
    "lattice decoder on every class tested", heavy=True)
```

```
def c10():
```

```
    rng = _rng()
```

```
    sample = [0]
```

```
    sample += [L2.class_of(REASONER.carrier(n))
```

```
               for n in REASONER.list_concepts()[:40]]
```

```
    sample += [rng.randrange(1, L2.N_CLASSES) for _ in range(40)]
```

```
    table = GR.type2_table()
```

```
    sample += list(table)[:20]
```

```
    agree = 0
```

```
    bad = []
```

```
    for c in sample:
```

```
        fast = REASONER.class_type(c)
```

```
        slow = L2.class_type(c)
```

```

if fast == slow:
    agree += 1
else:
    bad.append((hex(c), fast, slow))
ok = not bad
return ok, {"classes tested": len(sample), "agree": agree,
           "disagreements": bad[:5],
           "type-2 table size": len(table),
           "expected table size": 98280}

```

@claim("C11", "a type-2 class holds exactly 2 minimal vectors, a type-4 class "
 "holds 48 forming a coordinate frame of 24 orthogonal pairs")

```

def c11():
    table = GR.type2_table()
    cls2 = next(iter(table))
    v2 = L2.minimal_vectors_of_class(cls2)
    frame_cls = next(c for c in (L2.class_of(REASONER.carrier(n))
                                for n in REASONER.list_concepts()[:200])
                    if REASONER.class_type(c) == 4)
    fr = L2.frame_of_class(frame_cls)
    pairs = {}
    for v in fr:
        key = tuple(abs(int(t)) for t in v)
        pairs[key] = pairs.get(key, 0) + 1
    orthogonal = all(sum(a * b for a, b in zip(fr[i], fr[j])) in (0, -LAT.norm2(fr[i]))
                    for i in range(0, 6) for j in range(i + 1, 8))
    ok = (len(v2) == 2 and v2[0] == tuple(-t for t in v2[1])
          and len(fr) == 48 and orthogonal)
    return ok, {"minimal vectors of a type-2 class": len(v2),
               "they are +-lambda": v2[0] == tuple(-t for t in v2[1]),
               "vectors of a type-4 class": len(fr),
               "orthogonal pairs": len(fr) // 2,
               "sampled pairs are orthogonal or antipodal": orthogonal}

```

@claim("C12", "relative to a fixed minimal vector the 196,560 minimal vectors "
 "split by $|\lambda \cdot \mu|$ into $2 + 9,200 + 94,208 + 93,150$, so the "
 "type-2 CLASSES split as $1 + 4,600 + 47,104 + 46,575 = 98,280$ ")

```
def c12():
    cen = L2.pair_census()
    classes = {k: v // 2 for k, v in sorted(cen.items())}
    ok = (sum(cen.values()) == 196560
          and classes == {0: 46575, 1: 47104, 2: 4600, 4: 1}
          and sum(classes.values()) == 98280)
    return ok, {"vectors by  $|\lambda \cdot \mu|$ ": dict(sorted(cen.items())),
               "classes by  $|\lambda \cdot \mu|$ ": classes,
               "minimal vectors": sum(cen.values()),
               "type-2 classes": sum(classes.values())}
```

§4. $Q = 2^{(1+24)}$, THE COMPOSITION LAW

@claim("C13", "a symplectic basis of singular vectors exists and the cocycle "
 " $f(u,v) = \langle b_u, a_v \rangle$ satisfies $f(u,u) = q(u)$ and "
 " $f(u,v) + f(v,u) = B(u,v)$ ")

```
def c13():
    rep = XS.extraspecial_audit(full=False)
    ok = (rep["symplectic_basis_ok"] and rep["basis_spans"]
          and rep["cocycle_diagonal_is_q"] and rep["cocycle_polar_is_B"])
    return ok, {k: rep[k] for k in ("symplectic_basis_ok", "basis_spans",
                                   "cocycle_diagonal_is_q",
                                   "cocycle_polar_is_B", "cocycle_tests")}
```

@claim("C14", "the extraspecial relations hold: z is central of order 2, "
 " $x_u^2 = z^{q(u)}$, and $[x_u, x_v] = z^{B(u,v)}$ ")

```
def c14():
```

```

rep = XS.extraspecial_audit(full=False)
return bool(rep["all_relations_hold"]), rep["relations"]

```

```

@claim("C15", "|Q| = 2^25 and Q has 2^24 + 2^12 = 16,781,312 elements of order
"

```

```

    "at most 2, an independent confirmation of the plus type")

```

```

def c15():

```

```

    rep = XS.extraspecial_audit(full=False)
    inv = rep["involutions"]
    ok = (rep["order"]["matches"] and rep["involutions_match"]
          and inv["elements_of_order_at_most_2"] == (1 << 24) + (1 << 12))
    return ok, {"order": rep["order"], "involutions": inv,
               "2^24 + 2^12": (1 << 24) + (1 << 12)}

```

```

@claim("C16", "the 4096-dimensional Schrodinger representation is a faithful "
    "homomorphism in which the centre acts as -1")

```

```

def c16():

```

```

    rep = XS.extraspecial_audit(full=False)
    ok = (rep["rep_dim"] == 4096 and rep["z_acts_as_minus_one"]
          and rep["rep_is_homomorphism"] and rep["rep_is_faithful_on_sample"])
    return ok, {k: rep[k] for k in ("rep_dim", "z_acts_as_minus_one",
                                     "rep_is_homomorphism",
                                     "rep_is_faithful_on_sample")}

```

```

@claim("C17", "the Griess ledger closes: 196,884 = 300 + 98,280 + 24 x 4096,
"

```

```

    "and 196,884 is the q-coefficient of the j-function")

```

```

def c17():

```

```

    odd = 24 * XS.REP_DIM
    total = GR.DIM_A + GR.DIM_B + odd
    jseries = LAT.j_invariant_series(2)
    ok = (odd == 98304 and total == 196884 and jseries[2] == 196884)
    return ok, {"300 (symmetric matrices)": GR.DIM_A,

```

```
"98,280 (type-2 classes)": GR.DIM_B,
"24 x 4096": odd, "total": total,
"j-function q-coefficient": jseries[2]}
```

```
# =====
#
# §5. THE EVEN PART OF THE GRIESS ALGEBRA
# =====
```

```
@claim("C18", "the even part is built with dimension 300 + 98,280 = 98,580, "
        "commutative and non-associative, over exact Fractions")
def c18():
    rep = GR.griess_audit(full=False)
    ok = (rep["dimension"]["matches"] and rep["non_associative"]
          and rep["classes"] == 98280)
    return ok, {"dimension": rep["dimension"],
               "non-associative": rep["non_associative"],
               "a witness associator norm": str(rep["associator_norm"])}

```

```
@claim("C19", "the structure constants are DERIVED, not quoted: the identity "
        "acts as the identity and the form is Frobenius, which pins "
        "alpha = 1/2, beta = 1/4 and the form normalisation")
def c19():
    rep = GR.griess_audit(full=False)
    one = GR.identity()
    table = GR.type2_table()
    cls = next(iter(table))
    b = GR.b_vector(cls)
    m = GR.a_matrix(GR.outer(GR.class_representative(cls)))
    checks = {
        "identity acts as the identity": rep["identity_acts"],
        "form is Frobenius": rep["form_is_frobenius"],
        "1 . b = b": one.mul(b) == b,
    }

```

```

"1 . A = A": one.mul(m) == m,
"alpha": str(GR.ALPHA), "beta": str(GR.BETA),
"gamma": str(GR.GAMMA), "delta": str(GR.DELTA),
"(A1,A2) = 2 tr(A1 A2)": str(GR.FORM_A),
"(b,b) = 2": str(GR.FORM_B),
}
ok = (rep["identity_acts"] and rep["form_is_frobenius"]
      and one.mul(b) == b and one.mul(m) == m
      and (GR.ALPHA, GR.BETA, GR.GAMMA, GR.DELTA)
      == (F(1, 2), F(1, 4), F(1, 4), F(1, 4)))
return ok, checks

```

@claim("C20", "a_lambda = (1/8) P_lambda +- (1/2) b_lambda is idempotent with "

"(a, a) = 1, for both signs — a Majorana axis")

def c20():

```

rep = GR.griess_audit(full=False)
table = GR.type2_table()
cls = next(iter(table))
ap, am = GR.axis(cls, 1), GR.axis(cls, -1)
ok = (rep["axis_idempotent"] and rep["axis_minus_idempotent"]
      and str(rep["axis_norm"]) == "1"
      and ap.mul(ap) == ap and am.mul(am) == am
      and ap.form(ap) == 1 and am.form(am) == 1 and ap != am)
return ok, {"a+ idempotent": ap.mul(ap) == ap,
            "a- idempotent": am.mul(am) == am,
            "(a+, a+)": str(ap.form(ap)), "(a-, a-)": str(am.form(am)),
            "the two axes are distinct": ap != am,
            "axes available": GR.DIM_B}

```

@claim("C21", "ad(a) has spectrum exactly {1, 0, 1/4, 1/32} with dimensions "

"1 / 49,152 / 2,323 / 47,104, summing to 98,580")

def c21():

```

dims = GR.spectrum_dimensions()

```

```

ok = (dims["total"] == dims["expected"] == GR.DIM_EVEN
      and dims["1"] == 1 and dims["0"] == 49152
      and dims["1/4"] == 2323 and dims["1/32"] == 47104)
return ok, dims

```

@claim("C22", "those dimensions are FORCED by the pair census of C12 together "

"with the splitting $300 = 1 + 23 + 276$ of the matrix part")

```
def c22():
```

```

    classes = {k: v // 2 for k, v in L2.pair_census().items()}
    n0, n1, n2 = classes[0], classes[1], classes[2]
    dims = GR.spectrum_dimensions()
    predicted = {
        "1": 1,                # the axis itself, inside span{P, b}
        "1/32": n1,            # b_mu with |lambda.mu| = 1
        "1/4": n2 // 2 + 23,    # half of the |.|=2 pairs, plus lam(x)v
        "0": n0 + n2 // 2 + 276 + 1, # the rest, A" on lambda-perp, and the
                                   # complement of a inside span{P, b}
    }
    ok = (all(dims[k] == v for k, v in predicted.items())
          and 1 + 23 + 276 == GR.DIM_A - 0 and sum(predicted.values())
          == GR.DIM_EVEN)
    return ok, {"classes with |lambda.mu| = 0": n0,
               "classes with |lambda.mu| = 1": n1,
               "classes with |lambda.mu| = 2": n2,
               "predicted": predicted, "computed":
                   {k: dims[k] for k in ("1", "0", "1/4", "1/32")},
               "300 = 1 + 23 + 276": 1 + 23 + 276 == 300,
               "sum": sum(predicted.values())}

```

@claim("C23", "all six Ising fusion rules hold for a register axis, including "

"the graded ones $1/4 * 1/4$ in $1+0$ and $1/32 * 1/32$ in $1+0+1/4$ ")

```
def c23():
```

```
    rep = GR.griess_audit(full=False)["fusion"]
```

```

ok = bool(rep["all_rules_hold"] and rep["idempotent"]
          and rep["norm_one"] and rep["samples_are_eigenvectors"])
return ok, {"samples really are eigenvectors":
           rep["samples_are_eigenvectors"], **rep["rules"]}

```

@claim("C24", "the Miyamoto involution of a_λ is exactly the extraspecial "

"sign automorphism x_λ , and Q acts on the even part by "
 "algebra automorphisms preserving the form")

def c24():

```

rep = GR.griess_audit(full=False)
mi = rep["miyamoto"]
table = GR.type2_table()
it = iter(table)
mu = next(it)
x, y = GR.axis(next(it)), GR.axis(next(it))
gx = GR.apply_sign_automorphism(mu, x)
gy = GR.apply_sign_automorphism(mu, y)
equivariant = GR.apply_sign_automorphism(mu, x.mul(y)) == gx.mul(gy)
isometry = x.form(y) == gx.form(gy)
ok = (mi["miyamoto_is_extraspecial_sign"]
      and rep["extraspecial_is_automorphism"] and equivariant and isometry)
return ok, {"classes checked": mi["checked"],
           "Miyamoto = extraspecial sign": mi["miyamoto_is_extraspecial_sign"],
           "Q acts by automorphisms": rep["extraspecial_is_automorphism"],
           "sample:  $g(x.y) = g(x).g(y)$ ": equivariant,
           "sample: the form is preserved": isometry}

```

@claim("C25", "Norton-Sakuma types are DERIVED by closing the subalgebra: "

"2B (dim 2, ip 0), 4A (dim 5, ip 1/32), 2A (dim 3, ip 1/8) with "
 "the Sakuma identity $a.b = (1/8)(a + b - a_\rho)$ ", heavy=True)

def c25():

```

rep = GR.norton_sakuma_report()

```

```

orb = rep["orbits"]
ok = (rep["all_identified"]
      and orb[0]["type"] == "2B" and orb[0]["dimension"] == 2
      and orb[1]["type"] == "4A" and orb[1]["dimension"] == 5
      and orb[2]["type"] == "2A" and orb[2]["dimension"] == 3
      and orb[2]["sakuma_2A_identity"])
return ok, {f"|lambda.mu| = {k}": v for k, v in orb.items()}

```

```

# =====
#
# §6. THE MULTI-MOG-CUBE
# =====

```

```

@claim("C26", "all 4,096 Golay codewords are aligned to the MOG and every on
e "

```

```

      "casts a hexacode shadow in GF(4)^6")
def c26():
    rep = MG.mog_audit(full=False)["alignment"]
    ok = bool(rep["aligned"] and rep["failures"] == 0
              and rep["codewords_tested"] == 4096)
    return ok, {k: rep[k] for k in ("codewords_tested", "failures", "aligned")}

```

```

@claim("C27", "the three 4x2 bricks — the user's three 8-bit cubes — are "
      "octads, disjoint, covering all 24 cells: they form a TRIO")

```

```

def c27():
    rep = MG.mog_audit(full=False)["trio"]
    return bool(rep["is_a_trio"]), rep

```

```

@claim("C28", "the six MOG columns are tetrads whose pairwise unions are "
      "octads: they form a SEXTET, one of 1,771")

```

```

def c28():
    rep = MG.mog_audit(full=False)["sextet"]

```

```
return bool(rep["is_a_sextet"] and rep["sextet_count"] == 1771), rep
```

```
@claim("C29", "REFUTATION: the Golay trace on a cube is the even-weight "[8,7,2] code of size 128, NOT the Reed-Muller code RM(1,3); "  
"the shortened code is just {0, the cube}")
```

```
def c29():
```

```
    rep = MG.mog_audit(full=False)["cube_code"]  
    ok = (rep["trace_size"] == 128 and rep["trace_is_even_weight_code"]  
          and rep["shortened_size"] == 2  
          and rep["shortened_is_cube_and_zero"] and rep["rm13_claim_refuted"])  
    return ok, {**rep, "RM(1,3) would have size": 16}
```

```
@claim("C30", "the 30 octads disjoint from a cube cut the other 16 cells in "  
"the 30 hyperplanes of AG(4,2): 15 complementary pairs, closed "  
"under XOR, labelling the cells bijectively by  $F_2^4$ , with "  
"stabiliser AGL(4,2) of order 322,560")
```

```
def c30():
```

```
    rep = MG.mog_audit(full=False)["affine"]  
    ok = (rep["disjoint_octads"] == 30 and rep["hyperplanes"] == 30  
          and rep["complementary_pairs"] == 15  
          and rep["closed_under_addition"] and rep["labels_are_a_bijection"]  
          and rep["affine_group_order"] == 322560)  
    return ok, rep
```

```
@claim("C31", "the octad intersection census is {0:30, 2:448, 4:280, 8:1} and "  
"the design counts are 759 octads, 3,795 trios, 1,771 sextets; "  
"the octad stabiliser order 322,560 agrees with the M24 census")
```

```
def c31():
```

```
    rep = MG.mog_audit(full=True)  
    inter = rep["octad_intersections"]  
    cen = rep["census"]  
    m24 = rep["m24"]  
    ok = (inter == {0: 30, 2: 448, 4: 280, 8: 1})
```

```

and cen == {"trios": 3795, "octads": 759, "sextets": 1771}
and m24["octad_stabiliser_order"] == 322560
and m24["octad_stabiliser_is_AGL_4_2"]
and sum(inter.values()) == 759)
return ok, {"intersections with a fixed octad": inter,
    "designs": cen,
    "M24 order": m24["order"],
    "octad stabiliser": m24["octad_stabiliser_order"],
    "= AGL(4,2)": m24["octad_stabiliser_is_AGL_4_2"]}

```

```

@claim("C32", "the AMBIENT digit-plane stack of a Leech point IS the Leech "
    "congruences: plane 0 constant, plane 1 a Golay codeword casting "
    "a hexacode shadow, and a mod-8 condition on the coordinate sum")

```

```

def c32():
    rep = MG.mog_audit(full=False)["stack"]
    ok = (rep["plane0_constant"] and rep["plane1_is_golay"]
        and rep["plane1_casts_hexacode_shadow"] and rep["mod8_sum_condition
"]
        and rep["golay_codewords"] == 4096)
    return ok, rep

```

```

@claim("C33", "the two cube pictures address the same 24 cells consistently: "
    "cube coordinates are distinct and recover the MOG cell")

```

```

def c33():
    rep = MG.mog_audit(full=False)
    coords = [MG.cube_coordinates(i) for i in range(24)]
    ok = bool(rep["cube_addresses_distinct"]) and len(set(coords)) == 24
    return ok, {"distinct cube addresses": len(set(coords)),
        "sample (cell 0)": coords[0], "sample (cell 23)": coords[23]}

```

```

#

```

```

# §7. THE BRIDGE: A CONCEPT AS A WORD OF TEN MONSTER ADDRESSE

```

S

#

@claim("C34", "NEGATIVE RESULT: one reduction mod 2 is far too coarse — the "

"660 concepts land on only 9 classes of $\Lambda/2\Lambda$, and "
"even after dividing out the codec's common factor only 44")

def c34():

names = REASONER.list_concepts()

raw = {L2.class_of(REASONER.carrier(n)) for n in names}

prim = {L2.class_of(L2.primitive_point(REASONER.carrier(n)))
for n in names}

ok = len(raw) < 20 and len(prim) < 100 and len(names) >= 660

return ok, {

"concepts": len(names),

"distinct classes after one reduction mod 2": len(raw),

"distinct classes of the primitive point": len(prim),

"why": "the codec clears a common denominator, so slots are even",

"conclusion": "expand, do not reduce — see C35",

}

@claim("C35", "THE FIX: the 2-adic stack of ten $\Lambda/2\Lambda$ classes is "

"FAITHFUL — $\text{class_stack_rebuild}(\text{class_stack}(x)) == x$ for every "

"concept in the register")

def c35():

names = REASONER.list_concepts()

bad = []

stacks = set()

for n in names:

x = REASONER.carrier(n)

planes = REASONER.stack(n)

if L2.class_stack_rebuild(planes) != x:

bad.append(n)

stacks.add(tuple(planes))

```

carriers = {tuple(REASONER.carrier(n)) for n in names}
ok = not bad and len(stacks) == len(carriers)
return ok, {
    "concepts": len(names),
    "stack depth": L2.STACK_DEPTH,
    "offset": L2.STACK_OFFSET,
    "faithful for every concept": not bad,
    "failures": bad[:5],
    "distinct stacks": len(stacks),
    "distinct carriers": len(carriers),
    "stack <-> carrier is a bijection": len(stacks) == len(carriers),
}

```

```

@claim("C36", "the register's 6,600 planes distribute over the four types, and "
        "590 of the 660 concepts carry at least one Majorana 2A axis",
    heavy=True)

```

```

def c36():
    cen = REASONER.census()
    ok = (cen["unencodable"] == 0
          and sum(cen["plane_types"].values())
          == cen["concepts"] * cen["planes_per_concept"]
          and cen["concepts_with_an_axis"] > 500)
    return ok, cen

```

```

@claim("C37", "composing two meanings multiplies their extraspecial elements:
"

```

$x_u x_v = z^{f(u,v)} x_{\{u+v\}}$, with the phase equal to the cocycle")

```

def c37():
    pairs = [("mass", "speed"), ("force", "length"), ("energy", "time"),
             ("pressure", "volume"), ("charge", "voltage")]
    rows = {}
    good = True
    for a, b in pairs:
        try:

```

```

    r = REASONER.composition_is_group_law(a, b)
except Exception:
    continue
rows[f"{a} * {b}"] = (f"classes add {r['classes_add']}, "
                      f"group matches {r['group_matches']}, "
                      f"phase is the cocycle "
                      f"{r['phase_is_the_cocycle']}")
good = good and r["classes_add"] and r["group_matches"] \
    and r["phase_is_the_cocycle"]
w = REASONER.group_word("energy")
ok = good and bool(rows) and isinstance(w.u, int)
return ok, {**rows, "the word of energy in Q": str(w)}

```

```

@claim("C38", "two concepts have a ten-letter RELATION WORD over "
          "{1A, 2A, 4A, 2B}, and wherever both planes are axes the "
          "predicted inner product 1, 1/8, 1/32 or 0 holds in the algebra")

```

```

def c38():
    pairs = [("energy", "torque"), ("energy", "mass"), ("mass", "speed"),
             ("force", "pressure"), ("entropy", "energy")]
    rows = {}
    good = True
    checked = 0
    for a, b in pairs:
        r = REASONER.relation(a, b)
        rows[f"{a} vs {b}"] = (" ".join(r["relation_word"]))
            + f" ({r['checked_planes']} planes checked)"
        checked += r["checked_planes"]
    good = good and r["all_predictions_hold"]
    good = good and set(r["relation_word"]) <= {"1A", "2A", "4A", "2B"}
    ok = good and checked > 0
    return ok, {**rows, "axis-pair planes checked in the algebra": checked,
                "all predicted inner products hold": good}

```

```

@claim("C39", "the relation is symmetric, and 1A occurs exactly when the two "

```

"concepts share that plane — so the relation word is a genuine "
"invariant of the pair")

def c39():

names = REASONER.list_concepts()[:40]

symmetric = True

diagonal = True

for i, a in enumerate(names[:12]):

for b in names[i + 1:12]:

ra = REASONER.relation(a, b)["relation_word"]

rb = REASONER.relation(b, a)["relation_word"]

symmetric = symmetric and ra == rb

sa, sb = REASONER.stack(a), REASONER.stack(b)

for k, letter in enumerate(ra):

if (letter == "1A") != (sa[k] == sb[k]):

diagonal = False

cen = REASONER.relation_census(limit=40)

ok = symmetric and diagonal and sum(cen.values()) == 40 * 39 // 2

return ok, {"relation is symmetric": symmetric,

"1A exactly on shared planes": diagonal,

"plane-0 census over 40 concepts": cen,

"pairs": sum(cen.values())}

@claim("C40", "A 2A TRIANGLE IN THE REGISTER: two concepts in 2A position
on "

"plane 0 generate a 3-dimensional Norton-Sakuma algebra whose "

"third axis is the axis of their PRODUCT — the Monster carrying "

"out GLM composition", heavy=True)

def c40():

found = REASONER.find_triangle(limit=90)

if not found:

return False, {"error": "no 2A pair found in the register sample"}

tri = REASONER.triangle(*found)

ok = (tri["applicable"] and tri["third_class_is_the_product_concept"]

and tri["sakuma_identity"] and tri["subalgebra_dimension"] == 3

```

    and tri["inner_product"] == "1/8")
return ok, {"concepts": " , ".join(found), **{k: v for k, v in tri.items()
        if k not in ("a", "b")}}

```

```

@claim("C41", "the similarity (g(a), g(b)) is an exact rational and is "
        "invariant under the extraspecial group action")

```

```

def c41():
    pairs = [("energy", "speed"), ("energy", "force"), ("energy", "mass"),
            ("power", "energy"), ("entropy", "energy")]
    values = {}
    invariant = True
    mu = REASONER.stack("energy")[0]
    for a, b in pairs:
        ga, gb = REASONER.griess_vector(a), REASONER.griess_vector(b)
        s = ga.form(gb)
        values[f"({a}, {b})"] = str(s)
        ha = GR.apply_sign_automorphism(mu, ga)
        hb = GR.apply_sign_automorphism(mu, gb)
        invariant = invariant and ha.form(hb) == s
    near = REASONER.neighbours("energy", count=4, limit=120)
    ok = invariant and all(isinstance(v, str) for v in values.values())
    return ok, {**values, "invariant under x_mu": invariant,
        "nearest to energy": " , ".join(f"{n} ({s})" for n, s in near)}

```

```

@claim("C42", "a concept's own axis satisfies the Monster fusion law, and its "
        "plane gives an involution that two-colours the register")

```

```

def c42():
    fus = REASONER.fusion("energy")
    total = len(REASONER.list_concepts())
    orb = REASONER.involution_orbit("energy", plane=2, limit=total)
    flat = REASONER.involution_orbit("energy", plane=0, limit=total)
    ok = (fus["is_axis"] and fus["all_rules_hold"]
        and orb["fixed_count"] > 0 and orb["moved_count"] > 0
        and orb["fixed_count"] + orb["moved_count"] == total)

```

```

    and flat["moved_count"] == 0)
return ok, {"energy is an axis on plane": fus.get("plane"),
           "fusion rules": fus.get("rules"),
           "plane 2 involution fixes": orb["fixed_count"],
           "plane 2 involution negates": orb["moved_count"],
           "examples negated": " ", ".join(orb["moved"][:4]),
           "plane 0 alone is degenerate (fixes everything)":
           flat["moved_count"] == 0}

```

```

@claim("C43", "a type-4 plane of a concept resolves into a coordinate frame: "
        "48 lattice vectors in 24 orthogonal pairs")

```

```

def c43():
    fr = None
    for n in REASONER.list_concepts()[:60]:
        f = REASONER.frame(n)
        if f.get("has_frame"):
            fr = (n, f)
            break
    if fr is None:
        return False, {"error": "no type-4 plane in the sample"}
    n, f = fr
    ok = f["vectors"] == 48 and f["orthogonal_pairs"] == 24
    return ok, {"concept": n, "plane": f["plane"], "vectors": f["vectors"],
               "orthogonal_pairs": f["orthogonal_pairs"],
               "shape census": f["shape_census"]}

```

```

@claim("C44", "both MOG readings of a concept are available and agree: the "
        "ambient stack yields a Golay codeword with a hexacode shadow, "
        "the basis stack yields the Monster addresses")

```

```

def c44():
    m = REASONER.mog("energy")
    ok = (m["golay_plane_is_a_codeword"] and m["ambient_plane0_constant"]
          and len(m["basis_planes"]) == L2.STACK_DEPTH
          and len(m["hexacode_shadow"]) == 6)

```

```

    and len(m["golay_frame"]) == 4 and len(m["golay_frame"][0]) == 6)
return ok, {"golay plane": m["golay_plane"],
            "is a codeword": m["golay_plane_is_a_codeword"],
            "weight": m["golay_weight"],
            "hexacode shadow": m["hexacode_shadow"],
            "ambient plane 0 constant": m["ambient_plane0_constant"],
            "basis planes": len(m["basis_planes"]),
            "basis types": m["basis_types"]}

```

```

@claim("C47", "FACETS: each of the six groups of codec slots projects a "
            "concept to another point of Lambda, hence to another word of "
            "ten Monster addresses, and the six facet points sum back to "
            "the carrier")

```

```

def c47():

```

```

    from glm3_reasoner import FACET_SLOTS, ZERO_WORD
    names = REASONER.list_concepts()[:60]
    # (i) the six facets partition the 24 codec slots
    used = sorted(i for sl in FACET_SLOTS.values() for i in sl)
    partition = used == list(range(24))
    # (ii) every facet point is in Lambda and its word rebuilds it
    in_lambda = True
    rebuilds = True
    sums = True
    for n in names:
        total = [0] * 24
        for facet in FACET_SLOTS:
            x = REASONER.facet_point(n, facet)
            in_lambda = in_lambda and LAT.in_leech(list(x))
            w = REASONER.facet_word(n, facet)
            rebuilds = rebuilds and L2.class_stack_rebuild(w) == tuple(x)
            total = [a + b for a, b in zip(total, x)]
        sums = sums and tuple(total) == tuple(REASONER.carrier(n))
    # (iii) an unlabelled facet is exactly the word of the origin
    unlabelled = [f for f in FACET_SLOTS
                   if REASONER.facet_word("energy", f) == ZERO_WORD]

```

```

# (iv) the facets are informative: the dimension facet separates concepts
dim_words = {tuple(REASONER.facet_word(n, "dimension")) for n in names}
ok = (partition and in_lambda and rebuilds and sums
      and "context" in unlabelled and len(dim_words) > 1)
return ok, {
    "the six facets partition the 24 slots": partition,
    "every facet point is in Lambda": in_lambda,
    "every facet word rebuilds its point": rebuilds,
    "the facet points sum to the carrier": sums,
    "facets of 'energy' that are unlabelled": unlabelled,
    "distinct dimension words over 60 concepts": len(dim_words),
    "type word of the dimension facet of energy":
        REASONER.facet_report("energy")["dimension"]["type_word"],
}

```

```

@claim("C48", "an equation is DECIDED entirely inside the Monster: the facet "
        "words reproduce the GLM verdict on 7,140 library pairs and on "
        "a batch of expressions, and name the facet that failed",
      heavy=True)

```

```

def c48():
    import itertools
    names = REASONER.list_concepts()[:120]
    disagreements = []
    pairs = 0
    for a, b in itertools.combinations(names, 2):
        k = REASONER.monster_check(a, b)
        pairs += 1
        if not k["agrees_with_glm"]:
            disagreements.append((a, b))
    expressions = [
        ("mass*speed^2", "speed^2*mass", True),
        ("mass*speed^2", "mass*speed^4", False),
        ("force*length", "mass*speed^2", False),
        ("mass*acceleration", "force", True),
        ("mass*speed", "force", False),
    ]

```

```

("power*time", "energy", True),
("power*time", "force", False),
("energy/time", "power", True),
("energy*time", "power", False),
("length/time", "speed", True),
("length*time", "speed", False),
]
reports = [(a, b, want, REASONER.monster_check(a, b))
            for a, b, want in expressions]
expr_agree = all(k["agrees_with_glm"] for _, _, k in reports)
as_expected = all((k["verdict"] == "ADMISSIBLE") == want
                  for _, _, want, k in reports)
# the failing facet is named, and it is the right one
named = {f"{a} = {b}": k["failing_facets"]}
        for a, b, _, k in reports if k["failing_facets"]}
# plane 0 alone is strictly weaker
false_positives = [f"{a} = {b}" for a, b, _, k in reports
                   if k["mod2_false_positive"]]
classic = REASONER.monster_check("mass*speed^2", "mass*speed^4")
classic_ok = (classic["mod2_false_positive"]
              and classic["differing_planes"] == [3, 4, 5]
              and classic["failing_facets"] == ["dimension"])
ok = (not disagreements and expr_agree and as_expected
      and false_positives and classic_ok)
return ok, {
    "library pairs tested": pairs,
    "disagreements with the GLM verdict": len(disagreements),
    "expressions tested": len(expressions),
    "every expression verdict is the correct one": as_expected,
    "the failing facet is named": named,
    "plane 0 alone gives false positives": false_positives,
    "E = m c^4: failing facet": classic["failing_facets"],
    "E = m c^4: differing planes of the full carrier":
        classic["differing_planes"],
    "E = m c^4: relation word": classic["relation_word"],
}

```

@claim("C49", "ANALOGY as lattice arithmetic: $x_b - x_a + x_c$ is again a point "

"of Lambda, its ten addresses rebuild it, and ranking the "

"register against it recovers the intended answer", heavy=True)

def c49():

an = REASONER.analogy("mass", "force", "time", count=8)

in_lambda = bool(an["target_in_lambda"])

names = [n for n, _ in an["nearest"]]

force / mass is an acceleration, and acceleration * time is a speed:

the top of the ranking should be populated by the speed family

speedish = {"speed", "four_velocity", "alfven_speed", "ion_sound_speed",

"drift_velocity", "escape_velocity", "group_velocity",

"phase_velocity", "thermal_velocity", "terminal_velocity"}

hits = [n for n in names[:4] if n in speedish]

the construction is an identity when $b == a$

same = REASONER.analogy("mass", "mass", "time", count=1)

identity = "time" in same["exact"]

ok = in_lambda and len(hits) >= 3 and identity

return ok, {

"question": an["question"],

"target is a point of Lambda, rebuilt by its ten addresses": in_lambda,

"target plane types": an["target_types"],

"ranking": names[:5],

"speed-family concepts in the top four": hits,

"a : a :: c : ? returns c exactly": identity,

}

=====

§8. THE ARCHITECTURE INVARIANT

=====

```
@claim("C45", "THE ARCHITECTURE INVARIANT: the meaning is the state, the "
```

```
"carrier is derived from it, the Monster layer only READS the "  
"carrier, and no verdict is reached by reducing an exponent "  
"mod 2", heavy=True)
```

```
def c45():
```

```
    names = REASONER.list_concepts()  
    # (i) the carrier is encode(meaning), re-derived, for every concept  
    derived = all(tuple(REASONER.carrier(n))  
                  == tuple(encode(REASONER.meaning(n))) for n in names)  
    # (ii) no verdict object carries a mod-2 field  
    verdict = REASONER.audit("energy", "mass*speed^4")  
    no_mod2 = not (hasattr(verdict, "mod2_would_accept")  
                  or hasattr(verdict, "mod2_false_positive"))  
    no_mod2_text = "mod-2" not in str(verdict)  
    # (iii) the Monster layer is a pure function of the carrier: same carrier,  
    #       same address, and it never writes back  
    by_carrier: Dict[Tuple[int, ...], List[str]] = {}  
    for n in names:  
        by_carrier.setdefault(tuple(REASONER.carrier(n)), []).append(n)  
    shared = [v for v in by_carrier.values() if len(v) > 1]  
    pure = all(REASONER.stack(g[0]) == REASONER.stack(g[1]) for g in shared)  
    # (iv) the Monster layer changes no GLM verdict  
    unchanged = (bool(REASONER.audit("energy", "mass*speed^2").admissible)  
                 and not bool(verdict.admissible))  
    # (v) all algebra arithmetic is exact  
    exact = isinstance(REASONER.similarity("energy", "force"), F)  
    ok = (derived and no_mod2 and no_mod2_text and pure and unchanged  
         and exact)  
    return ok, {  
        "concepts": len(names),  
        "carrier = encode(meaning) for every concept": derived,  
        "no mod-2 verdict field": no_mod2,  
        "no mod-2 in the verdict text": no_mod2_text,  
        "groups of concepts sharing a carrier": len(shared),  
        "shared carrier => identical Monster address": pure,
```

```

    "GLM verdicts unchanged by the Monster layer": unchanged,
    "similarity is an exact Fraction": exact,
}

```

```

@claim("C46", "the one thing Section 14 says is NOT built is honestly not "
        "built, and the four that were are")

```

```

def c46():

```

```

    import glm3_odd as OD

```

```

    import glm3_metric as MET

```

```

    import glm3_sign as SGN

```

```

    # (b) the Monster itself is not generated: no sigma / triality generator

```

```

    no_monster = not any(hasattr(XS, name) for name in

```

```

        ("monster_generators", "sigma", "triality"))

```

```

    # (a) the odd part is built: both products exist and the ledger closes

```

```

    odd_built = (hasattr(OD, "act") and hasattr(OD, "product")

```

```

        and OD.ledger()["agrees"])

```

```

    # (c) the sign is settled, by convention and by the odd part

```

```

    sign_settled = (SGN.CANONICAL_SIGN == -1

```

```

        and OD.sign_visibility_report(count=1)[

```

```

            "the_odd_part_separates_the_two_signs"])

```

```

    # (d) the depth is derived

```

```

    depth_derived = (L2.derive_stack_parameters(180, L2.STACK_OFFSET)[1]

```

```

        == L2.STACK_DEPTH)

```

```

    # (e) the distance is a metric

```

```

    metric = MET.positive_definite_report()["all_ok"]

```

```

    # the two axes of a class are still both valid idempotents

```

```

    table = GR.type2_table()

```

```

    cls = next(iter(table))

```

```

    ap, am = GR.axis(cls, 1), GR.axis(cls, -1)

```

```

    both_valid = (ap.mul(ap) == ap and am.mul(am) == am

```

```

        and ap.form(ap) == am.form(am) == 1 and ap != am)

```

```

    ok = (no_monster and odd_built and sign_settled and depth_derived

```

```

        and metric and both_valid)

```

```

    return ok, {

```

```

        "(a) the odd part is built": odd_built,

```

```

"(b) the Monster itself is not generated": no_monster,
"(c) the sign of an axis is settled": sign_settled,
"(d) the depth is derived": depth_derived,
"(e) the distance is a metric": metric,
"both signs are still valid idempotents": both_valid,
"what IS built": "Q = 2^(1+24) acting by automorphisms on a genuine "
    "196,884-dimensional Griess algebra",
}

```

```

# =====
=====

```

```

# §9. THE DEPTH IS DERIVED

```

```

# =====
=====

```

```

@claim("C50", "the stack depth is DERIVED, not chosen: the register's "
    "coordinate range is 180, the least admissible pair is "
    "(offset 256, depth 9), and offset 512 forces depth 10")

```

```

def c50():
    names = REASONER.list_concepts()
    points = [REASONER.carrier(n) for n in names]
    rep = L2.depth_report(points, extra_depths=2)
    ok = (rep["coordinate_range"] == 180
        and (rep["least_offset"], rep["least_depth"]) == (256, 9)
        and rep["depth_forced_by_the_module_offset"] == L2.STACK_DEPTH
        and rep["module_depth_is_the_derived_one"])
    return ok, {
        "points": rep["points"],
        "coordinate range of the register": rep["coordinate_range"],
        "least admissible (offset, depth)": (rep["least_offset"],
            rep["least_depth"]),
        "module offset": rep["module_offset"],
        "depth it forces": rep["depth_forced_by_the_module_offset"],
        "module depth": rep["module_depth"],
        "the module depth is the derived one":

```

```

    rep["module_depth_is_the_derived_one"],
}

```

```

@claim("C51", "PROPOSITION D1: rebuild(stack(x)) = x at EVERY admissible "
    "(offset, depth) pair, and above the threshold the extra "
    "planes are identically zero while the lower planes do not "
    "move")

```

```

def c51():
    names = REASONER.list_concepts()
    points = [REASONER.carrier(n) for n in names]
    rep = L2.depth_report(points, extra_depths=3)
    # and on points that are not concepts: the origin and a scaled point
    extra = [tuple([0] * 24), tuple(3 * c for c in points[0])]
    off, dep = L2.derive_stack_parameters(L2.coordinate_range(extra))
    extras_ok = all(L2.stack_is_faithful(p, dep, off) for p in extra)
    ok = (rep["faithful_everywhere"] and rep["deeper_planes_are_zero"]
        and rep["lower_planes_unchanged"] and extras_ok)
    return ok, {
        "(offset, depth) pairs tried": len(rep["faithful"]),
        "faithful at every pair": rep["faithful_everywhere"],
        "deeper planes are identically zero": rep["deeper_planes_are_zero"],
        "lower planes unchanged": rep["lower_planes_unchanged"],
        "faithful off the register too": extras_ok,
        "pairs": sorted(rep["faithful"]),
    }

```

```

@claim("C52", "a concept's word at depth 12 is its word at depth 10 followed "
    "by two zero planes, so nothing above the threshold carries "
    "information")

```

```

def c52():
    from glm3_reasoner import MonsterReasoner
    deep = MonsterReasoner(depth=L2.STACK_DEPTH + 2)
    names = REASONER.list_concepts()[:120]
    same = True

```

```

padded = True
for n in names:
    base = REASONER.stack(n)
    more = deep.stack(n)
    same = same and more[:L2.STACK_DEPTH] == base
    padded = padded and not any(more[L2.STACK_DEPTH:])
return same and padded, {
    "concepts": len(names),
    "the first ten planes agree": same,
    "the extra planes are zero": padded,
    "type word of energy at depth 10":
        "".join(str(REASONER.class_type(p))
                 for p in REASONER.stack("energy")),
    "type word of energy at depth 12":
        "".join(str(deep.class_type(p)) for p in deep.stack("energy")),
}

```

@claim("C53", "the reasoning is DEPTH-INDEPENDENT above the threshold: a "

"reasoner at (offset 256, depth 9) and one at (offset 512, "
 "depth 12) return the same verdict on every pair", heavy=True)

```

def c53():
    import itertools
    from glm3_reasoner import MonsterReasoner
    shallow = MonsterReasoner(depth=9, offset=256)
    deep = MonsterReasoner(depth=12, offset=512)
    names = REASONER.list_concepts()[:60]
    disagree = []
    pairs = 0
    for a, b in itertools.combinations(names, 2):
        va = shallow.monster_check(a, b)["verdict"]
        vb = deep.monster_check(a, b)["verdict"]
        vc = REASONER.monster_check(a, b)["verdict"]
        pairs += 1
        if not (va == vb == vc):

```

```

    disagree.append((a, b, va, vb, vc))
# the similarity is a function of the meaning, not of the parameters
sims = {shallow.similarity("energy", "speed"),
        deep.similarity("energy", "speed"),
        REASONER.similarity("energy", "speed")}
ok = not disagree and len(sims) == 1
return ok, {
    "pairs": pairs,
    "parameter sets": [(9, 256), (12, 512), (L2.STACK_DEPTH,
                                                L2.STACK_OFFSET)],
    "disagreements": len(disagree),
    "examples": disagree[:3],
    "similarity(energy, speed) at every depth": str(sims.pop()),
}

```

```

# =====
# §10. THE CANONICAL SIGN
# =====

```

```

@claim("C54", "the Golay theta function  $\theta(C) = |C|/4 \bmod 2$  is quadratic "
    "and not linear, and PROPOSITION S2:  $q([2 \cdot 1_C]) = \theta(C)$  "
    "for all 4,096 codewords")

```

```

def c54():
    import glm3_sign as SGN
    t = SGN.theta_report(full=True)
    leech = SGN.theta_is_the_leech_form()
    ok = (t["quadratic_identity"] and not t["is_linear"]
        and leech["q_equals_theta"])
    return ok, {
        "codewords": t["codewords"],
        "theta(C+D) = theta(C)+theta(D)+|C&D|/2": t["quadratic_identity"],
        "theta is linear": t["is_linear"],
        "codewords checked against q": leech["codewords_checked"],
    }

```

```

    "q([2 . 1_C]) = theta(C) everywhere": leech["q_equals_theta"],
}

```

```

@claim("C55", "PROPOSITION S1: the Sakuma identity FORCES s(lambda+mu)
= "

```

```

    "-s(lambda)s(mu), so the all-plus convention is incoherent and "
    "s = -1 is the canonical coherent one; the coherent "
    "conventions number 2^24")

```

```

def c55():

```

```

    import glm3_sign as SGN
    coh = SGN.coherence_report()
    con = SGN.conventions_report(dimension=16)
    sak = SGN.sakuma_report()
    ok = (coh["rule_holds_everywhere"]
        and not coh["all_plus_convention_is_coherent"]
        and coh["canonical_convention_is_coherent"]
        and con["nullity_is_24"]
        and con["every_B_convention_solves_the_system"]
        and sak["all_hold"])

```

```

    return ok, {
        "triangles checked": coh["triangles"],
        "s(lambda+mu) = -s(lambda)s(mu) everywhere":
            coh["rule_holds_everywhere"],
        "the all-plus convention is coherent":
            coh["all_plus_convention_is_coherent"],
        "the canonical convention is coherent":
            coh["canonical_convention_is_coherent"],
        "canonical sign": SGN.CANONICAL_SIGN,
        "closed subsystem": (con["subspace_dimension"],
                             con["classes_in_the_subsystem"]),
        "nullity of the triangle system": con["nullity"],
        "coherent conventions": con["number_of_coherent_conventions"],
        "Sakuma holds with no ad-hoc sign": sak["all_hold"],

```

```

    }

```

#

§11. THE ODD PART

#

@claim("C56", "the odd part $V^- = 24 \times 4096$ is built with its four constants "
"DERIVED from the identity condition and Miyamoto, and the "
"ledger $1 / 96,256 / 4,371 / 96,256 = 196,884$ is the classical "
"2A one")

def c56():

import glm3_odd as OD

c = OD.derive_constants()

led = OD.ledger()

block = OD.spectrum_block()

ok = (c["identity_condition"] and c["over_determined_system_closes"]
and c["matches_module_constants"]
and c["counting_puts_one_quarter_on_the_along_block"]
and block["eigenvalues_are_the_monster_set"]
and block["total"] == OD.DIM_ODD and led["agrees"]
and OD.DIM_FULL == 196884)

return ok, {

"dimensions": f"{GR.DIM_EVEN} + {OD.DIM_ODD} = {OD.DIM_FULL}",

"c1, c2, c3, c4": (str(c["c1"]), str(c["c2"]), str(c["c3"]),
str(c["c4"])),

"identity acts as the identity": c["identity_condition"],

"the over-determined system closes":

c["over_determined_system_closes"],

"counting puts 1/4 on the 2,048-dimensional block":

c["counting_puts_one_quarter_on_the_along_block"],

"eigenspace dimensions on V^- ": block["dimensions"],

"even part": led["even"],

"whole algebra": led["whole"],

"these are the classical 2A numbers": led["agrees"],

}

```
@claim("C57", "both products of the odd part are exact: commutative, "
    "Frobenius (which is how  $V^- \times V^- \rightarrow V^+$  is DEFINED), and "
    "equivariant for Q, and every block of the axis spectrum is "
    "an explicit eigenvector")
```

```
def c57():
```

```
    import glm3_odd as OD
    comm = OD.commutativity_report(trials=4)
    frob = OD.frobenius_report(trials=3)
    equi = OD.equivariance_report(trials=3)
    spec = OD.spectrum_report(count=2)
    ok = (comm["commutative"] and frob["frobenius_holds"]
        and equi["equivariant"] and spec["all_are_eigenvectors"]
        and spec["eigenvalues_are_the_monster_set"])
    return ok, {
        "commutative": comm["commutative"],
        "Frobenius  $(u.v, w)^+ = (w|>u, v)^-$ ": frob["frobenius_holds"],
        "Frobenius checks": frob["checks"],
        "product is Q-equivariant": equi["product_is_equivariant"],
        "action is Q-equivariant": equi["action_is_equivariant"],
        "explicit eigenvectors": len(spec["rows"]),
        "eigenvalues seen on  $V^-$ ": spec["eigenvalues_seen"],
    }
```

```
@claim("C58", "the fusion rules that involve the odd part hold, including "
    " $1/32 * 1/32 \rightarrow 1 + 0 + 1/4$ , and the odd part SEPARATES the "
    "two axes of a class:  $\tau(a^-) = x_\lambda$ ,  $\tau(a^+) = "$ 
    " $x_\lambda z$ ")
```

```
def c58():
```

```
    import glm3_odd as OD
    fus = OD.fusion_report(count=2)
    miy = OD.miyamoto_report(count=2)
    sgn = OD.sign_visibility_report(count=2)
    ok = (fus["all_rules_hold"]
```

```

    and miy["tau_is_the_extraspecial_sign_on_the_odd_part"]
    and sgn["the_odd_part_separates_the_two_signs"])
return ok, {
    "fusion checks": fus["checks"],
    "all fusion rules hold": fus["all_rules_hold"],
    "Miyamoto checks": miy["checks"],
    "tau(a^-)": miy["tau_of_a_minus"],
    "tau(a^+)": miy["tau_of_a_plus"],
    "the odd part separates the two signs":
        sgn["the_odd_part_separates_the_two_signs"],
    "eigenvalues of a^- and a^+ on lambda (x) s":
        [(r["eigenvalue_of_a_minus"], r["eigenvalue_of_a_plus"])
         for r in sgn["rows"]],
    "the even part cannot": True,
}

```

```

# =====
#
# §12. THE METRIC
# =====
# =====

```

```

@claim("C59", "the invariant form is POSITIVE DEFINITE on the even part — it "
        "is twice a sum of squares — so the Griess distance is a "
        "pseudometric and the triangle inequality is free")
def c59():
    import glm3_metric as MET
    pd = MET.positive_definite_report()
    names = REASONER.list_concepts()[:40]
    vectors = {n: REASONER.griess_vector(n) for n in names}
    rng = _rng()
    triangle = True
    for _ in range(200):
        a, b, c = (rng.choice(names), rng.choice(names), rng.choice(names))
        if not MET.triangle_holds(

```

```

        MET.pseudo_distance2(vectors[a], vectors[b]),
        MET.pseudo_distance2(vectors[b], vectors[c]),
        MET.pseudo_distance2(vectors[a], vectors[c])):
    triangle = False
    break
quot = MET.quotient_report(
    {n: REASONER.griess_vector(n) for n in REASONER.list_concepts()})
ok = (pd["all_ok"] and pd["both_coefficients_positive"] and triangle
      and quot["distance_separates_classes"])
return ok, {
    "form constants (A, b)": (pd["form_a"], pd["form_b"]),
    "both positive": pd["both_coefficients_positive"],
    "the form is a sum of squares in the (A, B) basis":
        pd["sum_of_squares"],
    "triangle inequality on 200 triples": triangle,
    "concepts": quot["concepts"],
    "distinct Griess vectors": quot["classes"],
    "concepts with no axis at all": quot["no_axis_at_all"],
    "largest fibre": quot["largest_fibre"],
    "a metric on the quotient": quot["distance_separates_classes"],
}

```

@claim("C60", "the plane-graded embedding is INJECTIVE, so the distance is a "

"true metric on the register: distance zero exactly for "

"concepts sharing a carrier", heavy=True)

def c60():

```

import glm3_metric as MET
names = REASONER.list_concepts()
stacks = {n: REASONER.stack(n) for n in names}
inj = MET.injectivity_report(stacks)
sep = MET.separation_report(stacks)
audit = MET.metric_audit(stacks=None, vectors=None)
ok = (inj["is_a_metric_on_the_register"]
      and audit["plane_vector_injective_on_sample"]

```

```

    and audit["plane_vector_zero_only_at_zero"]
    and audit["fast_distance_agrees_with_the_algebra"])
return ok, {
    "concepts": inj["concepts"],
    "distinct stacks": inj["distinct_stacks"],
    "pairs at distance zero with different carriers":
        inj["distance_zero_yet_different"],
    "pairs at positive distance with equal carriers":
        inj["distance_positive_yet_equal"],
    "a metric on the register": inj["is_a_metric_on_the_register"],
    "the plane vector is injective":
        audit["plane_vector_injective_on_sample"],
    "the O(24) distance agrees with the algebra":
        audit["fast_distance_agrees_with_the_algebra"],
    "closest pair": (sep["closest_pair"], sep["closest_distance"]),
    "farthest pair": (sep["farthest_pair"], sep["farthest_distance"]),
    "nearest neighbours of energy":
        [n for n, _ in MET.nearest("energy", stacks, 4)],
}

```

```

# =====
#
# §13. THE BENCHMARK
# =====

```

```

@claim("C61", "BENCHMARK A: the exhaustive pairwise sweep of all 217,470 "
    "pairs of the register — the Monster verdict and the GLM "
    "verdict never disagree", heavy=True)
def c61():
    import glm3_bench as BM
    rep = BM.pairwise_sweep()
    ok = (rep["pass_rate"] == 1 and rep["disagreements"] == 0
        and rep["unrolled_form_agrees"] and rep["pairs"] >= 217000)
    return ok, {

```

```

"concepts": rep["concepts"],
"pairs": rep["pairs"],
"agreements": rep["agreements"],
"disagreements": rep["disagreements"],
"admissible": rep["admissible_pairs"],
"rejected": rep["rejected_pairs"],
"cross-checked against monster_check":
    rep["cross_checked_against_monster_check"],
"the unrolled form is the same computation":
    rep["unrolled_form_agrees"],
"pass rate": str(rep["pass_rate"]),
}

```

```

@claim("C62", "BENCHMARK B: a corpus of real physical laws — Newton, "
    "Coulomb, Ohm, Planck, Stefan-Boltzmann, Bernoulli, Lorentz, "
    "Friedmann and the rest — is admissible, every one")

```

```

def c62():
    import glm3_bench as BM
    rep = BM.law_report()
    ok = (rep["pass_rate"] == 1
        and rep["monster_agrees_with_glm"] == rep["laws"]
        and not rep["errors"])
    return ok, {
        "laws": rep["laws"],
        "admissible": rep["admissible"],
        "Monster agrees with GLM": rep["monster_agrees_with_glm"],
        "failures": rep["failures"],
        "errors": rep["errors"],
        "pass rate": str(rep["pass_rate"]),
        "examples": [r["law"] for r in rep["rows"][:8]],
    }

```

```

@claim("C63", "BENCHMARK C: every law corrupted four ways — exponent, sw
ap, "

```

"rank, scale — is caught, with a zero false-negative rate, and "
"the facet that caught it is the facet the mutation moved")

def c63():

```
import glm3_bench as BM
rep = BM.mutant_report()
attribution = rep["facet_attribution"]
predicted = (
    all(k.startswith("dimension") for k in attribution["exponent"])
    and all(k.startswith("dimension") for k in attribution["swap"])
    and set(attribution["rank"]) == {"tensor"}
    and set(attribution["scale"]) == {"scale"})
ok = (rep["pass_rate"] == 1 and not rep["false_negatives"]
      and not rep["monster_glm_disagreements"] and predicted)
return ok, {
    "mutants": rep["mutants"],
    "vacuous (the meaning did not change)": rep["vacuous"],
    "genuine corruptions": rep["genuine_corruptions"],
    "rejected by verdict": rep["rejected_by_verdict"],
    "refused by the parser": rep["refused_by_the_parser"],
    "false negatives": len(rep["false_negatives"]),
    "false-negative rate": str(rep["false_negative_rate"]),
    "caught by the stack but not by plane 0":
        rep["caught_by_the_stack_but_not_by_plane_0"],
    "facet attribution": attribution,
    "the attribution is the predicted one": predicted,
    "pass rate": str(rep["pass_rate"]),
}
```

@claim("C64", "BENCHMARK D: the dimensionless groups — Reynolds, Mach, "
"Prandtl, Nusselt, Froude, Weber, Peclet and the rest — come "
"out dimensionless, and the two textbook Coriolis groups are "
"reported as carrying a radian rather than hidden")

def c64():

```
import glm3_bench as BM
rep = BM.dimensionless_report()
```

```

ok = (rep["pass_rate"] == 1 and not rep["failures"]
      and rep["dimensionless"] >= rep["groups"] - 2)
return ok, {
    "groups": rep["groups"],
    "as expected": rep["as_expected"],
    "exactly dimensionless": rep["dimensionless"],
    "carrying a radian": rep["carrying_a_radian"],
    "failures": rep["failures"],
    "pass rate": str(rep["pass_rate"]),
}

```

```

# =====
=====
# RUNNER
# =====
=====

```

```

def run(quick: bool = False, verbose: bool = True) -> Dict[str, object]:
    started = time.time()
    results: List[Dict[str, object]] = []
    passed = 0
    skipped = 0
    for cid, statement, fn, heavy in CLAIMS:
        if quick and heavy:
            skipped += 1
            results.append({"id": cid, "statement": statement,
                           "status": "skipped"})
        if verbose:
            print(f" {cid:5s} SKIP {statement}")
            continue
        t0 = time.time()
        try:
            ok, detail = fn()
        except Exception as exc:
            # pragma: no cover
            ok, detail = False, {"exception": repr(exc)}

```

```

dt = time.time() - t0
passed += 1 if ok else 0
results.append({"id": cid, "statement": statement,
               "status": "PASS" if ok else "FAIL",
               "seconds": round(dt, 3), "detail": _jsonable(detail)})
if verbose:
    print(f" {cid:5s} {'PASS' if ok else 'FAIL'} {statement}")
    for k, v in _flatten(detail):
        print(f"          {k:46s} {v}")
elapsed = time.time() - started
total = len(CLAIMS) - skipped
summary = {
    "claims": len(CLAIMS),
    "run": total,
    "passed": passed,
    "failed": total - passed,
    "skipped": skipped,
    "seconds": round(elapsed, 2),
    "results": results,
}
if verbose:
    print()
    print(f" {passed}/{total} claims pass"
          + (f", {skipped} skipped" if skipped else "")
          + f", in {elapsed:.1f} s")
return summary

```

```

def _jsonable(obj):
    if isinstance(obj, dict):
        return {str(k): _jsonable(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [_jsonable(v) for v in obj]
    if isinstance(obj, F):
        return str(obj)
    if isinstance(obj, (int, float, str, bool)) or obj is None:

```

```
    return obj
return str(obj)
```

```
def _flatten(detail, prefix=""):
    out = []
    if isinstance(detail, dict):
        for k, v in detail.items():
            if isinstance(v, dict):
                out.extend(_flatten(v, f"{prefix}{k}."))
            else:
                out.append((f"{prefix}{k}", _short(v)))
    else:
        out.append((prefix or "value", _short(detail)))
    return out
```

```
def _short(v):
    s = str(v)
    return s if len(s) <= 92 else s[:89] + "..."
```

```
def main(argv: Sequence[str]) -> int:
    quick = "--quick" in argv
    json_only = "--json" in argv
    if not json_only:
        print("=" * 80)
        print(" THE GEOMETRIC LANGUAGE MACHINE, THIRD GENERATION")
        print(" reasoning inside the Monster — verification run"
              + (" (quick)" if quick else ""))
        print("=" * 80)
        print()
    summary = run(quick=quick, verbose=not json_only)
    here = os.path.dirname(os.path.abspath(__file__))
    outdir = os.path.join(here, "results")
    os.makedirs(outdir, exist_ok=True)
```

```
path = os.path.join(outdir, "glm3_results.json")
with open(path, "w") as fh:
    json.dump(summary, fh, indent=2, sort_keys=False)
if not json_only:
    print(f" results written to {os.path.relpath(path, here)}")
return 0 if summary["failed"] == 0 else 1
```

```
if __name__ == "__main__": # pragma: no cover
    sys.exit(main(sys.argv[1:]))
```